



Copyright Statement

This copy was made pursuant to the [Fair Dealing Advisory of the University of Waterloo](#) and/or is covered and authorized by a license granted by the owners of the copyright on the works reproduced therein.

The copy may only be used for the purpose of research, private study, criticism, review, news reporting, education, satire or parody. If the copy is used for the purpose of review, criticism, or news reporting, the source and the name of the author must be mentioned. The use of this copy for any other purpose may constitute violation of Canadian Copyright Law, and/or relevant licensing agreements.

Structure of the Eh Compiler:
Lexical Scanning, Syntactic Analysis and Optimization

Gary J. Stafford

A thesis
presented to the University of Waterloo
in partial fulfillment of the requirements
for the degree of
Master of Mathematics
in
Computer Science

Waterloo, Ontario 1978

(c) Gary J. Stafford, 1978

PROPERTY OF THE LIBRARY
UNIVERSITY OF WATERLOO

I hereby declare I am the sole author of this thesis.

I authorize the University of Waterloo to lend this thesis to other institutions or individuals for the purpose of scholarly research.

A handwritten signature in cursive script, appearing to read "Sun Hui", written in dark ink.

I further authorize the University of Waterloo to reproduce this thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

A handwritten signature in cursive script, appearing to read "Sun Hui", written in dark ink.

The University of Waterloo requires the signatures of all persons using or photocopying this thesis. Please sign below and give address and date.

Abstract

By structuring the compiler for the Eh language as several phases, which communicate through intermediate languages, it is possible to perform the various tasks of compilation independently of each other. After the lexical scanning phase of the compiler, the source being compiled has been translated to a string of tokens. The parsing phase converts the string of tokens into an intermediate language described by an LR(0) grammar. The optimization phase checks all function calls to make sure that the correct number of arguments are passed. These first three phases are used when compiling for any machine. They thus provide syntactic portability for an Eh program. The remaining phases produce loader directives for specific machines and are different for each machine.

Size is a major consideration in the compiler since it is expected to execute on mini-computers. Because of this, whenever a decision had to be made between size and speed, the smaller solution was usually taken.

Acknowledgements

This work would not have been possible without the support and direction of Prof. Michael A. Malcolm. The facilities provided by the University of Waterloo Mathematics Faculty Computing Facility were also extremely helpful. People in the Portability Group provided a great deal of help in designing and debugging the compiler. Included in this group are: Bert Bonkowski, Reinaldo Braga, John Corman, Dave Cheriton, Morven Gentleman, Tom Miller, Alfredo Piquer, Patricio Poblete, Gary Sager, Ian Telford, and Fred Young.

Table of Contents

1.	Introduction	1
2.	Overall Structure of the Compiler	3
3.	Lexical Scan	9
3.1	Files Used	10
3.2	Symbol Table	12
3.3	Structure	19
3.4	Input	20
3.4.1	File Inclusion	21
3.4.2	Conditional Exclusion	22
3.4.3	Manifest Definition	24
3.4.4	Manifest Expansion	25
3.5	Token Analysis	25
3.5.1	Constant Evaluation	27
3.5.2	Symbolic Names	29
3.5.3	Literal Strings	30
3.6	Termination	31
3.7	Lexil	33
3.7.1	Tokens File	34
3.7.2	Strings File	37
4.	Syntactic Analysis	39
4.1	Symbol Table	40
4.2	Expression Trees	42
4.3	Constant Folding	46
4.4	Error Recovery	50
4.5	Parsil	52
4.5.1	The Parsil Grammar	53
4.5.2	The Externals File	54
4.5.3	Functions File	56
4.5.4	Points of Interest	57
5.	Function Modification	59
5.1	Structure	60
5.2	Input	61
5.2.1	Module Definition	62
5.2.2	Function Input	63
5.3	Structure Modification	63
5.3.1	Unreachable Statements	65
5.3.2	Straighten Transfers	67
5.3.3	Unreferenced Labels	69
5.3.4	Null Transfers	70
5.3.5	Conditional to Expression	71
5.3.6	Remove Useless Expressions	73
5.3.7	Reverse Conditional Transfers	73

5.4	Expression Modification	75
5.4.1	Sethi-Ullman Balance	76
5.4.2	Swapped Operators	77
5.4.3	Fetch Operators	78
5.4.4	Modify Operators	80
5.4.5	Remove Useless Operands	82
5.5	Output	86
5.6	Library Searching	88
5.7	Function Call Checks and Modification	89
5.8	Cleanil	90
6.	Conclusions	91
6.1	Future Work	93
	Bibliography	95
A.	Lexical Scanner	97
B.	Parsil Grammar	103
C.	Eh Grammar	107
D.	Cleanil Grammar	113
E.	An Example	117
E.1	Source Code	118
E.2	Output of Lex	120
E.3	Output of Parse	122
E.4	Output of Clean	125

List of Illustrations

2.1	Compiler structure	8
3.1	Lex symbol table node	15
4.1	Parse symbol table node	41
4.2	Binary operator tree node	42
4.3	Unary operator tree node	43
4.4	Nullary operator tree node	43
4.5	Constant operand tree node	44
4.6	Non-constant operand tree node	44
5.1a	Subscript expression for value	79
5.1b	Fetch operator added to subscript	79
5.2a	Indirect of plus expression	80
5.2b	Indirect of plus changed to subscript	80
5.3a	Address of subscript expression	81
5.3b	Address of a subscript modified to a plus	81
5.4a	Expression with useless constant 0 operand	83
5.4b	Expression replaced by left operand	83
5.5a	Expression with useless non-zero operand	84
5.5b	Expression replaced by constant 0	84
5.6	Function call record	86

List of Tables

3.1 Symbol table performance	16
3.2 One-byte token representations	36
3.3 Source location markers	37

Chapter 1

Introduction

This thesis deals with the lexical scan, syntactic analysis, and optimization phases of the Eh compiler, as well as the intermediate languages used by these phases. Eh is a descendant of the B language, developed at Bell Laboratories, Johnson (1973). B is a descendant of BCPL, Richards (1969). The compiler is designed to run under the Thoth operating system Cheriton et. al, (1978). The overall structure of the compiler is briefly described, a detailed examination of parts of the compiler is presented, and certain aspects which contribute to its simplicity and value are discussed.

Large parts of the compiler, reading the source, recognizing tokens, maintaining symbol tables, and modifying control paths and expressions, can be performed with little knowledge of the target machine.

Since the Eh compiler is a multi-phase compiler the time needed for the compilation is greatly influenced by the sizes of the intermediate representations used to communicate between phases. These representations have been made concise.

Both novel and standard ideas have been incorporated into the Eh compiler with the result that this thesis should be of interest to both those who know little about compiling and those with much experience in the field. Some of the ideas presented here can be used in many programs having nothing to do with compiling. More can be of use in compilers for other languages. Most of the lexical scanner for example, is also used as the lexical scanner of an assembler.

Chapter 2 examines the overall structure of the compiler. Each subsequent chapter presents a specific phase of the compiler and the intermediate language produced by that phase. The phases are presented in the order in which they are executed.

Some of the more interesting aspects of the compiler are: a secondary file containing symbolic names which is used by all phases, LR(0) intermediate languages which use Polish-postfix expression representations, and syntactic checks on all compiled function calls.

Chapter 2

Overall Structure of the Compiler

Before going further two definitions are needed. The host machine is the machine on which the compiler is executing. The target machine is the machine for which the program is being compiled.

The Eh compiler is divided into 5 distinct phases: **Lex**, **Parse**, **Clean**, **Data**, and **Code**. Figure 2.1 shows the structure in a graphic form. The **Source** files are read by **Lex**, which produces a file containing the **Lexil** version of the source and a second file which contains symbolic strings. The **Lexil** file is read by **Parse** which again outputs two files; functions in one, and externals in the other. The functions file is processed by **Clean** to produce an improved (optimized) representation of the functions. The externals file from **Parse** is read by **Data** which outputs load code modules. Load code modules are also generated by **Code**. Of the 5 phases only **Data** and **Code**, Bonkowski (1978), are specific to the target machine.

Lex reads the source, follows file inclusions, expands manifests, and handles conditional exclusions. It identifies the tokens used, and produces a concise representation which is easier for **Parse** to manipulate. **Lex** and **Parse**, originally a single phase, were split to reduce space requirements. One effect of the split is that a program has to be completely scanned for tokens before any syntactic analysis is done. This can lead to the lexical scanner spending a significant amount of time reading before the parser indicates the first syntax error.

Lex outputs two files, one containing an encoded version of the input in the intermediate language **Lexil**, and a second file containing copies of the symbolic names of all identifiers found in the source, copies of all literal strings, and the pathnames of all files which contain the source.

Lexil is an intermediate language which is used to communicate between **Lex** and **Parse**. While preserving all the necessary information found in the source, it presents the parser with a simplified input which has token representations of a predefined length. All keywords and operators are encoded in a single byte. Symbolic names and literal strings are encoded as three bytes, one of which

identifies the representation as a symbolic name or literal string, and two that contain the byte index in the Strings file where a copy of the symbolic name or literal string is stored. This index is termed the id of the name or string. Constants are represented by a variable number of bytes, the number of bytes being specified in the first byte used to encode the constant. **Parse** can also check for the equivalence of any two symbolic names simply by checking the equivalence of the two ids.

Parse analyses the **Lexil** output of **Lex** for syntactic correctness on a module by module basis. Constant expressions are reduced to simple constants. A second intermediate representation of the input program is produced in a language called **Parsil**. The single input stream from **Lex** is converted into two output streams, one for functions and the other for externals.

Parsil is a language which is easily parsed by the two phases which read it. It is a statement based language. All control structures in the Eh source have been simplified into simple conditional or unconditional transfers. Since all **Parsil** programs are produced by **Parse**, they are all syntactically correct. This guarantee of syntactic correctness means that processing these programs is much

simpler than processing programs in languages which might not be syntactically correct. Certain assumptions can be made by the phases which read **Parsil** programs and these assumptions will always be true. A major benefit of **Parsil** is that all statements are identified by their first byte, making recognition simple.

Clean simplifies control paths, modifies expressions to simplify the task of code generation, and performs checks on function calls. It produces a modified version of the output of **Parse** which, while more concise, contains more information. The output is in the intermediate language called **Cleanil**. Because **Clean** has space for a complete module in memory it can perform analyses and modifications more easily and efficiently than can **Parse**.

Cleanil, while resembling **Parsil**, contains a conditional transfer statement which is not available in **Parsil**. It is this extra statement, and the richer set of operators which **Clean** produces, that can make the **Cleanil** representation more concise than is possible in **Parsil**.

Data takes the **Externals** output of **Parse** and converts it into load code directives. Because of the simplicity of its task **Data** is a small and efficient phase in the compilation process. It is this simplicity which allows

Data, while being specific to a given target machine, to be easily changed to handle the externals for a different target machine.

Code takes the output of **Clean** and converts it into load code directives. The most complex of all the phases, **Code** is the only phase which deals with the instruction set of the target machine. **Code** is the phase on which most of the effort is spent when porting the compiler.

Lex, **Parse** and **Clean** provide the syntactic portability for Eh. Since lexical scanning, parsing and optimization are done by the same programs for all target machines, any Eh program is guaranteed to be syntactically portable. Also, since these phases are written in Eh in a portable manner, the same source for these three phases can be compiled for, and run on, all host machines. There is only one **Lex**, one **Parse**, and one **Clean**. This guarantee of syntactic portability thus holds over all host machines, subject to the caveat that the maximum allowable size of a module may differ from host to host.

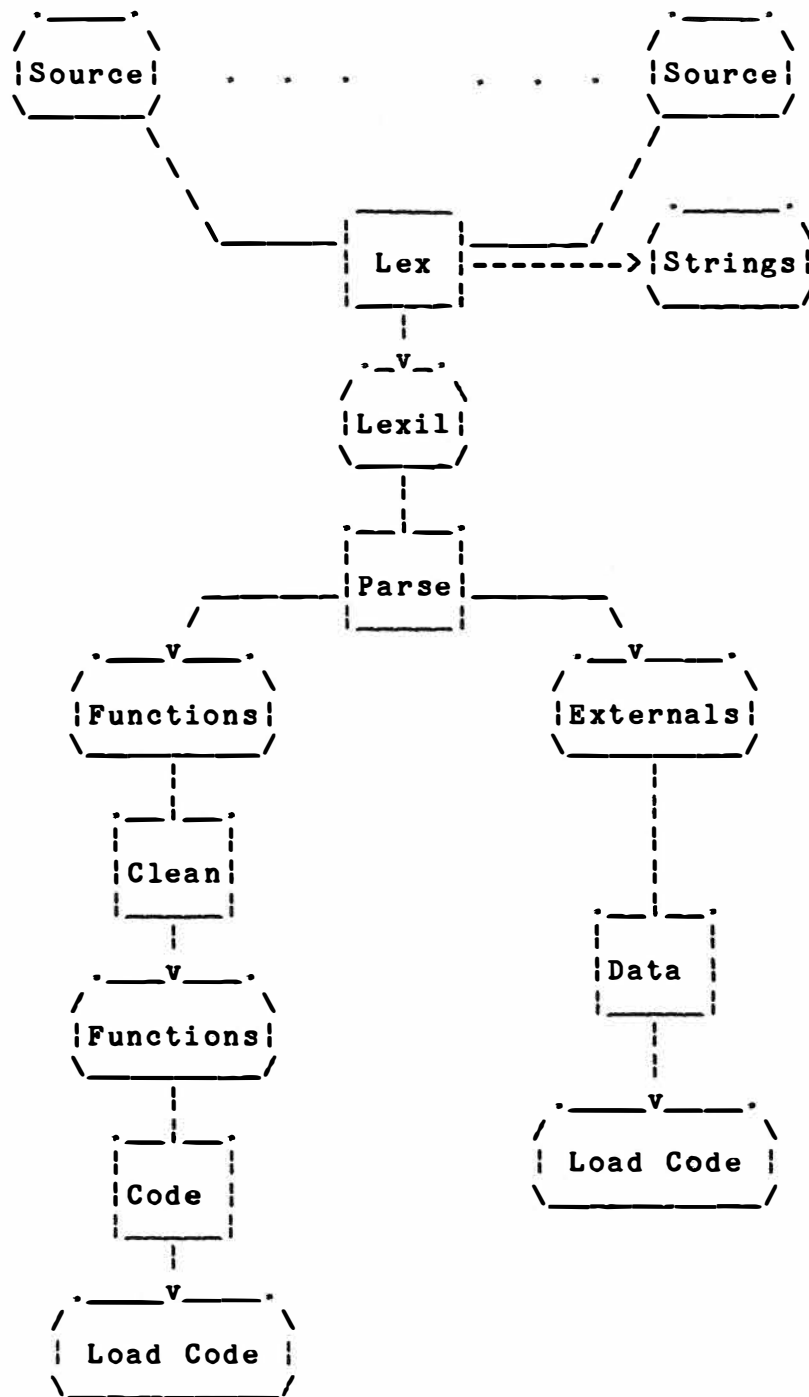


Figure 2.1 : Compiler structure

Chapter 3

Lexical Scan

Lex has two major responsibilities. One is the recognition of the tokens of the language found in the user's source. The other is determining which characters are seen by the recognizer. These two tasks are relatively independent. By separating token analysis from control of the input the token analyser can assume a single source of input.

Tables are used when scanning constants and symbolic names. By using the input character as an index into a table valid alphabetic and numeric characters can be recognized efficiently and, in the case of numerics, converted to the value the character represents.

Lex is the least complex of the first three compiler phases. Although its code size is small it requires a large data area for the symbol table.

3.1 Files Used

Lex produces two output files, **Tokens** and **Strings**. It makes use of one work file, **Manifests**. The **Tokens** file is read by **Parse**. The **Strings** file is read by all phases of the compiler other than **Lex**.

The **Manifests** file holds the definitions of manifests. Due to the large number of manifests defined in some Eh programs, the expense in size incurred in keeping the text in memory is prohibitive. It may in fact make compilation impossible. Hence the use of the **Manifests** file. While the saving in space may seem to be offset by a loss in time used to access the manifest text, statistics collected from compilations have shown that a large percentage of all defined manifests are seldom referenced. A manifest tends to be used, on the average, slightly less than twice. For example, manifests used to define a symbol table node tend to be used only in the functions which search for or insert elements into the symbol table.

The **Tokens** file contains the stream of tokens produced by **Lex**. Since it contains a representation of the program being compiled it will be examined in conjunction with the Lexil intermediate language.

The Strings file holds symbolic names, pathnames of input files, and literal strings. **Data** and **Code** need symbolic names in order to correctly generate data for the loader or library editor, while **Parse** and **Clean** only need them if an error message is produced. **Data** and **Code**, in fact, do not themselves make use of the names, but only copy them into load code files at the appropriate time.

Pathnames are held in the Strings file so that, if an error or warning message is printed by later phases, a specific line in a specific file can be indicated.

Literal strings are stored in the Strings file for the same reasons as symbolic names. It is not until **Data** or **Code** that the characters composing the strings are needed. **Parse** and **Clean** need only know that a string was present in the source. The Strings file will be described in detail in a later section when **Lexil** is discussed.

3.2 Symbol Table

A symbolic name is either a keyword, a manifest name, or an identifier. The name is first tested to see if it is a keyword. To speed testing, the first two characters of the name are compared to the first two characters of the keywords to find a possible match. Only if the first two characters match are the symbolic names compared for equality. Since the definition of Eh states that there are at least 16 bits in a word and since ASCII characters fit in 7 bits the two character test can be performed by a simple word-select statement. This reduces the number of string comparisons needed, and consequently speeds the detection of keywords. Note that except for the keywords "repeat" and "return", all keywords are unique in the first two characters.

If the symbolic name is not a keyword then a hash value is computed, yielding an index into a 64-entry hash table. Symbolic names are stored in linked lists which are rooted at these hash buckets. The name being searched for is compared with each name in the linked list. Prior to a complete comparison a second key is checked. The key consists of the first word in the symbolic name. A word comparison is much less expensive in execution time than a

string comparison. The word comparison will work on all host machines for two reasons. First, when a symbolic name is encountered the first word of the storage locations used to hold the name is initialized to 0. Second, the area used in the symbol table to store the name is allocated in words and these words are all initialized to 0. This second key was chosen chiefly because it requires no extra space in the symbol table node. Since **Lex** stores all the symbolic names in main memory the saving of one word per node is very important.

If the name is not found it is inserted at the head of the list. If the name is found the node which corresponds to the name is moved to the head of the list. This scheme is the "self organizing list" discussed in Knuth (1973).

Each node in the symbol table contains four fields: **LINK**, **TYPE**, **LOCATION**, and **NAME**. See Figure 3.1.

The **LINK** field contains a pointer to the next node in the list. The last node on a list has a **LINK** field which contains 0.

The 1-word **TYPE** field contains either a zero signifying that the node represents an identifier, 256 signifying that the node represents a manifest, or a value between 0 and 256

which specifies that the node represents a keyword. While keywords are not stored in the body of the symbol table they are stored in nodes which are structured in the same way as symbol table nodes.

The **LOCATION** field contains a number which has one of two meanings. If the node represents an identifier the **LOCATION** field contains the Strings file id of the stored symbolic name. If the node represents a manifest the **LOCATION** field contains the location in the Manifests file where the associated text may be found.

The **NAME** field contains the symbolic name associated with the node. Since symbolic names can be up to 32 characters in length, a large amount of space could be wasted if enough space were reserved for 32 characters in each node. Thus the field is of variable length, just big enough to hold the symbolic name. While it would be possible to use the copy of the name stored in the Strings file the cost of accessing the file for every name found in the source, not just every new name, would be prohibitive.

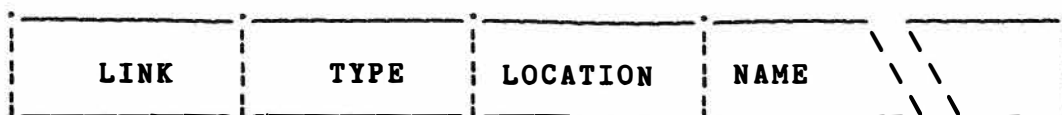


Figure 3.1 : Lex symbol table node

The symbol table representation and search scheme may not be the best possible. However measurements have shown that only 3 to 4 percent of the execution time in **Lex**, on the average, is spent searching the symbol table. Table 3.1 shows the symbol table performance for 10 arbitrarily chosen programs. The programs range from a null program (`Main();`) to the **Code** phase for the TI/990 Eh compiler. Note that there are 42 environmental manifests defined for all compilations, making the minimum number of symbols 43. Since the comparison of symbolic names is the most expensive part of the search, the number of such comparisons is a valid measure of the success of a symbol table scheme. With the present scheme 25 to 50 percent of the time spent searching the symbol table is spent in string comparisons. The column labeled "Optimum" shows the number of string comparisons that would have to be done if for every new symbol no comparison was made, and, for every symbol already in the table one comparison was made to verify the name.

Number of Symbols	Number of References	Number of Comparisons (Average)	Number of String Comparisons (Average)	Optimum
742	9477	3.27	.93	.92
508	3300	2.06	1.09	.84
279	2308	1.97	.89	.87
273	773	1.38	.89	.64
188	457	2.04	.68	.58
174	208	1.57	.44	.19
66	74	1.01	.52	.10
63	67	.97	.56	.05
60	62	1.04	.43	.03
43	43	.35	.00	.00

Table 3.1: Symbol table performance

There is no guarantee that the scheme chosen will perform well for all programs. It is interesting to consider a worst-case program. If all names have same first character, same last character, and same length, all names will hash to the same bucket. The names should have the same first few characters, so that the second key algorithm is defeated. The names should also be used cyclicly; i.e. when a name is used it is not used again until all other names have been used. The values in the "Number of Symbols" column in Table 3.1 are close to the worst-case average numbers of string comparisons.

Program-generated programs can defeat the hashing function. Other hash functions could have been chosen, but for every hash function there is a set of identifiers which will be handled poorly.

Program-generated programs can also have names which are the same in the first few characters, making the second key useless. There is no inexpensive solution to this problem. If an alternate method of obtaining the second key from the symbolic name has to be used, the cost of each comparison will likely increase.

The third possibility, cyclic use of names, is not so easy to attain. Names used for functions will tend to be used infrequently and so will settle to the tail of the list. Local variables normally are used more than once in a function and will settle near the head of the list in their respective functions.

The data in the second row of Table 3.1 comes from compiling the parser phase of the Eh compiler. All function names created by the parser generator start with the characters "Act" and all externals start with the characters "Yy". Every generated function references all the generated externals in the same order, an approximation to cyclic use. The numbers in figure 3.1 are not as good as for the other

programs but they are still quite acceptable.

Various alternative symbol table schemes were considered. Binary trees or heaps require either larger symbol table nodes or larger areas in which to hold the nodes. Keeping the lists in alphabetically sorted order was tried but it was found that the average number of comparisons was greater. The cost of comparison for an alphabetically sorted list is also greater since not only inequality, but the direction of inequality, has to be computed.

Given that the compilation will not continue if too many symbols are defined, how many symbols can **Lex** be expected to handle? While **Eh** identifiers can be up to 32 characters in length, the average has been found to be 9. On a machine with two bytes per word the average name thus occupies 5 words. The rest of the symbol node is 3 words in length yielding an average of 8 words per node. With a symbol table of 10000 words, 1250 symbols could be stored. On a host with 4 bytes per word this number would rise to 1666 symbols. A rather complex program would be needed to exceed this; the **Code** phase for the TI/990 **Eh** compiler is a substantial program but contains only 742 symbols. 10000 words is reasonable for the size of the symbol table since,

with the code needed and all other data, the final size of **Lex** is approximately 17000 words.

Finally one might ask what happens when the symbol table space is exhausted. When the space required exceeds the space available **Lex** prints an appropriate message and terminates. The user has then to compile the program in parts, store them in a library, and load the program from the library. To date, this has happened only when the symbol table size was artificially made small during testing.

3.3 Structure

As mentioned earlier, **Lex** can be divided into two distinct sections. One handles input of characters from the source and the other performs token analysis.

The interface presented by the input section is a call to a function which returns the next character in the input stream. All necessary manipulations of the input are hidden from the code which does the token analysis.

Token analysis is under the control of a single function. Any additional processing, such as searching for names in the symbol table, or evaluating a constant, is typically handled by other small functions.

3.4 Input

The Eh compiler considers source to be composed of both statements to be compiled and commands to the scanner. These commands tell the scanner where to find more source, how to treat the source that it has found, and what parts of the source are to be compiled.

All lines which are commands must have a special character in the first column of the input line. The special characters are '?' '%' '#' and '@'. Each of these characters requests a different action and these actions are discussed in the following sections. The simple first-character rule is complicated slightly by the fact that literal strings are allowed to cross the end of an input line and hence include the first character of the next line. The rule that is actually used is, "If a literal string is not being collected, and the first character on an input line marks the line as a command, the line contains a command".

3.4.1 File Inclusion

File inclusion is a means of having the compiler obtain source from a file other than the one currently being read. The '%' character is used to request a file inclusion. The '%' is followed by the pathname of the file which is to be included. The pathname is optionally followed by one or two constants which define relocation base registers. These relocation base registers will be ignored in further discussions. If further information is desired, see Sager (1978).

When file inclusion is requested, appropriate information about the current file is stacked. When the included file has been processed, the stack top indicates how to continue processing the original input.

When entering an included file care is taken to see that the line number is initialized correctly in case errors are detected. When the included file is opened and selected for reading the current line number is set to 0 and the existence of an end of line character is simulated. This is done since the recognition of the start of a line is keyed on the knowledge that the last character read was the end of a line. The simulated end of line character keeps the line

numbers correct, and allows any command on the first line of a file to be recognized.

There is one other **Lex** command which has to do with file inclusions. A command specified by '@' requests that the compiler change its current node. The '@' must be followed by the pathname of the new current node. Use of file inclusion allows users to keep files which contain relative pathnames as inclusion files. The major advantage of relative pathnames is a substantial increase in speed of compilation. For small files with long pathnames it can take longer for the file system to find the file than for **Lex** to process it.

3.4.2 Conditional Exclusion

Conditional exclusion is used by the programmer to have the compiler ignore parts of the source. All conditional exclusion commands are specified by a '?' followed by one of three keywords, **ifdef**, **else** or **end**. The keyword **ifdef** is followed by a symbolic name. These conditional control commands may be nested. The commands are used in the form:


```
?ifdef symbolic name
```

```
    source to be excluded if symbolic name is not  
    defined
```

```
?else
```

```
    source to be excluded if symbolic name defined
```

```
?end
```

or, if there is no need for the else segment:

```
?ifdef symbolic name
```

```
    source to be excluded if symbolic name is not  
    defined
```

```
?end
```

Two variables are used to keep track of conditional exclusion. One contains the level of nesting and the other contains the number of levels of nesting which are being ignored. Encountering an **ifdef** increments the level of nesting and encountering an **end** decrements it. If the level of nesting being ignored is non-zero an **ifdef** increments it. An **end** decrements the level of nesting being ignored if it is non-zero. The **else** command will only affect the level of nesting being ignored, and only when the level is 0 or 1, changing it to 1 or 0 respectively.

3.4.3 Manifest Definition

A command with a '#' in column one specifies the command as a manifest definition. The '#' is followed by any valid Eh identifier, followed by a '=', followed by a sequence of characters terminated by either an end-of-line character or a comment delimiter (\). The identifier is the manifest name and the sequence of characters which follow the '=' is the manifest text. The manifest name and text are recorded and all future occurrences of the manifest name found in the input will effectively be replaced with the manifest text.

A blank is appended to the end of the stored text to facilitate checking for multi-character tokens crossing the end of a manifest expansion. Without this blank, detection of a token crossing the end of a manifest expansion would be impossible because the token analyser is not aware of the end of the manifest text. Since the input functions have to be aware of the collecting of characters for literal strings, any attempt to have a literal string cross the end of a manifest text can be easily detected. Since the only case in which the blank can be significant is an error this solution is quite acceptable.

3.4.4 Manifest Expansion

Manifest expansion is one point where the token analysis and input control sections of **Lex** interact. Because a manifest name can be any Eh identifier, the symbolic name must be encountered and recognized as a symbolic name before it can be recognized as a manifest name. The token analysis function which recognizes the manifest name calls a function which belongs to the input section to have the manifest expanded through actions similar to those which occur when a file inclusion is requested. The next character read will come from the manifest text. When the manifest text has been completely read input returns to the previous input level.

3.5 Token Analysis

The structure of the analyser is based on repeatedly selecting an action on the next character returned from the input section.

Some thought was given to making the token analysis table-driven, however the select statement in the function compiles to a jump table which is very efficient. Another argument put forth for table-driven analysis is that it is

easy to change the tables to accept new tokens. This has some merit when the language is being developed. At later dates in the life of the language representations of the tokens might also change. This change in representations happened to Eh: the assignment operators had their representations swapped e.g. "=" became "+=", etc. The select statement caused the change to take 2 hours while the change using tables might have taken only 15 minutes. This sort of change does not take place very often however. The major reason for not using a table driven token analyser is historical. An almost identical version of the main analyser function was in existence before a generator was available for easily building the tables, and when one became available it was decided that the effort to convert would be better spent on other areas.

Most of the token representations in Eh are more than one character in length, thus identification of the first character only lowers the number of possible tokens. The first character will indicate whether an operator or operand has been encountered. If an operator is indicated further identification is simple. Once a set of operators has been identified by the first character, it is a relatively small task to correctly identify the operator. No character which starts a multi-character operator has more than two

different characters which can follow it. Therefore, a simple "if ... else ..." construct is sufficient for identifying all multi-character operators. When the operator has been identified a different more compact representation of the operator is placed in the Tokens file.

Symbolic names, numeric constants, character constants and strings are handled differently. A special function is called for each of these types of operands since collection of the characters representing these operands is more complex than for operators. As well, further manipulations are needed for operands which are not necessary for operators. Because of these extra complications operands will be covered in more depth.

3.5.1 Constant Evaluation

When a constant is encountered it is read and the value is computed. A constant can be specified in one of three bases, octal, decimal, or hexadecimal. To simplify the code for recognition of each digit an initialized vector is used. The vector is 128 words in length. To find if a character is valid in specifying a constant in a given base, the character is used to index into the vector, and the value of

that word is compared to the base of the constant. If the value is less than the base then the character is valid. Since the vector is initialized so that the character indexes a word containing the value the character represents, the value is also available. Thus recognition and evaluation of each digit is not only simple but very efficient.

After each digit is evaluated it is combined with the constant so far collected. The constant already assembled is multiplied by the base and the new digit added in. Constants are evaluated and stored in a vector of length 128 words. The first word of the vector contains the number of words of the vector that are needed to store the constant. The other words each contain 8 bits of the value of the constant. Multiplication by the base, and addition of the new digit, is done byte-serially. While byte-serial multiplication sounds expensive, data collected from 38 different programs shows that 99.6 percent of the constants scanned require one 8-bit byte for representation. In these cases a maximum of one multiplication need be performed. By keeping track of the number of bytes needed to contain the value of the constant, only the minimum number of multiplications is done.

Character constants are handled similarly however, since it is known that there is only one "digit" involved, there is no need for byte serial multiplication. One complication arises when the character constant is represented by an escape sequence which specifies the constant by three octal digits. This can produce 9 significant bits, which requires two 8-bit bytes for accurate representation.

3.5.2 Symbolic Names

When an alphabetic character is found by the scanner, the rest of the symbolic name has to be collected. Here we are presented with the same problem as in constant digit recognition, and the same solution is taken. The vector used for digit recognition is also used for alphanumeric recognition. Since the characters of the name do not have to be translated, the value obtained from the vector is simply used to check for validity. The collection of symbolic names is thus simple and efficient. After the name has been collected it is either found in or inserted into the symbol table.

3.5.3 Literal Strings

When a literal string is encountered, the characters which make up the string are copied to the Strings file. The evaluation is also simple. Simple characters retain their ASCII values. Any escaped characters are converted to their respective binary values. The length of the string is stored in the Strings file preceding the copy of the string.

The actual string is placed in the Strings file to speed and simplify the **Parse** and **Clean** phases. Since a large amount of the time taken by these phases is spent in input and output, decreasing the number of characters read or written improves performance. Their complexity is also lowered since they do not have to know how to process literal strings of arbitrary length. The performance of **Data** and **Code** is also improved since these phases do not have to scan the contents of the strings, only copy the correct number of characters to their output files.

3.6 Termination

While the lexical scanner of Eh is not a macro processor it has many things in common with one. One of the problems which many "text expanders" can encounter is a failure to terminate. The possibility of an infinite loop exists whenever input can be directed to an object which has already been input.

The claim made is that the lexical scanner will terminate with all inputs. One point must be made first. In the Thoth file system, reading past the last character written into a file returns a null character.

The main scanning function, as presented in Appendix A, will advance at least one character through the source for every call to the input function. In any function which this main function calls, all loops which do not input characters will terminate, and all loops which do input characters will terminate on a null character. Thus for any given source of input, the scanner will eventually read a null, and will return to the previous source of input.

The above argument is sufficient, provided there is not a recursive file inclusion or manifest expansion. For file inclusions limiting the number of stacked input files is sufficient, since a file inclusion command is terminated by an end of line character, which is not processed until the inclusion is finished. Thus it is impossible to recursively include files without stacking the input.

Recursive manifest expansions are also guaranteed to stack the input due to the blank at the end of the manifest text, which was mentioned in section 3.4.3. Since collecting the characters of a symbolic name requires reading one character past the symbolic name, not having the blank present could lead to an infinite loop. In collecting the name from the text all of the text would be read, input popped to the previous level for the one extra character, and the collected name, identified as a manifest, used to again direct input from the manifest text. The scanner would be caught between level N where the manifest was originally used, and level $N+1$ where the text is read. The blank at the end of the text assures that any manifest expansion which starts while a manifest text is being expanded, will terminate before the end of the original expansion. Thus, recursively expanding a manifest will always stack the input, and the recursion will be stopped when the maximum

stacking is encountered.

3.7 Lexil

Lexil is the language used to communicate, to **Parse**, the string of tokens being compiled. Two files are used to contain the representation of **Lexil**. The major file is called **Tokens**; it contains all the information pertinent to the structure of the input. The second file, **Strings**, contains copies of the symbolic forms of identifiers and literal strings found in the input.

The grammatical structure of **Lexil** is that of **Eh**. The representation of **Lexil** is important and shall be discussed in depth.

A token in **Lexil** represents a terminal symbol in the **Eh** grammar. A source location marker is an indicator which is used to associate the tokens of **Lexil** with the corresponding source as provided by the programmer. A marker may fall between any two tokens. Markers preserve the format of the source program and allow later phases to specify the line and the file in which an error was detected.

3.7.1 Tokens File

The Tokens file is the major output file of Lex, containing the Lexil representation of the source program. The best way to describe the Tokens file is to describe its elements.

Each token is represented by a sequence of bytes, as are the source location markers. Most tokens are represented by one byte, however identifiers and strings are represented by three, and constants are represented by one to 128 bytes. Table 3.2 lists the tokens that require only one byte for representation, and the value of the byte representing them.

Identifier and string token representations contain one byte which identifies the token as an identifier or string, followed by two bytes which contain the Strings file id.

The first byte of a constant representation has a value greater than 127. This byte, modulo 128, specifies how many bytes follow, containing the value of the constant, most significant byte first.

Source location markers either specify that a new line has been started, or that a file has been entered on a specified line. The start of line marker requires only one

byte. The marker for the entry into a file requires six bytes. The first byte specifies that a switch in input file has been encountered. The next two bytes contain the Strings file id of the identifying pathname. The next two bytes contain the line number of the file at which entry is made. If the line number is 0 the file is being entered for the first time. If the line number is non-zero the file has been reentered after having been left to include another file. The last byte is used to contain the relocation base register names as mentioned in section 3.4.1. With this form of file entry marker, later phases of the compiler only have to remember the current file and line number. Table 3.3 lists the markers and gives the values of their first bytes.

<u>Token</u>	<u>Value of byte</u>	<u>Token</u>	<u>Value of byte</u>
=	\$01	{	\$20
/=	\$02	}	\$21
*=	\$03	,	\$22
%=	\$04	;	\$23
+=	\$05	?	\$24
-=	\$06	!!	\$25
>>=	\$07	&&	\$26
<<=	\$08	+	\$27
&=	\$09	-	\$28
=	\$0A	*	\$29
^=	\$0B	&	\$2A
==	\$0C	::	\$2C
!=	\$0D	:	\$2D
>	\$0E	#	\$2E
>=	\$0F	auto	\$30
<	\$10	extrn	\$31
<=	\$11	if	\$35
>>	\$12	else	\$36
<<	\$13	for	\$37
!	\$14	repeat	\$38
^	\$15	while	\$39
/	\$16	select	\$3A
%	\$17	break	\$3B
++	\$18	goto	\$3C
--	\$19	next	\$3D
~	\$1A	return	\$3E
!	\$1B	case	\$3F
(	\$1C	default	\$40
)	\$1D	enable	\$41
[	\$1E	disable	\$42
]	\$1F	twit	\$43
		active	\$44

Table 3.2: One-byte token representations

Marker	Value of first byte
-----	-----
START_OF_LINE	\$7E
START_OF_FILE	\$7F

Table 3.3: Source location markers

3.7.2 Strings File

There are three different types of string kept in the Strings file; identifier names, literal strings, and pathnames of source files.

Identifiers and pathnames are simply stored as the symbolic characters used by the programmer. They are terminated by a null character.

Literal strings are slightly more complicated. Since the programmer can specify any character within a literal string, there can be no special character which will unambiguously delimit a string. The solution chosen was to store the length with the string.

The Strings file contains some strings which are not taken from the source provided to the compiler. One literal string and two identifiers are always present in the Strings file. At location 0 of the Strings file there is a null string stored. When **Parse** finds a string external that is not initialized, it specifies this null string as the initializer. The two identifiers always present are **.Nargs** and **.Arg**. Since these two identifiers are at predetermined locations later phases which wish to know if these identifiers are used can simply check to see if the id used to identify a symbolic name is equal to one of these two values. **Clean** needs to know if either of these two functions are called to correctly identify the type of function call required.

Chapter 4

Syntactic Analysis

Parse is based on a YACC parser, Johnson (1974). When the parser detects the right-hand side of a grammatical production it calls a semantic function specified in the grammar definition. The grammar of Eh, as given to YACC, is presented in Appendix C. The semantic functions have been removed for brevity. It is not the purpose of this section to go into details of the parsing method. Instead the auxiliary functions necessary to change a recognizer into a translator will be discussed.

Input is parsed one module at a time. Data structures are built while processing a module and then removed at the end of the module.

The expression tree structure and error recovery are similar to those in other compilers. The constant folding however is of interest. Constants are folded with the precision of the target machine, a source of many difficulties. The symbol table is also interesting mainly due to the form of the input given to **Parse** by **Lex**.

4.1 Symbol Table

The symbol table uses a 64-bucket hash table like that used by **Lex**. The search key is not the symbolic name but the **Strings** file id. The hash value is the least significant 6 bits of the search key. The self-organizing hash chains used by **Lex** are not used since the number of symbols in one given module is usually small. Data collected from 8 arbitrary programs shows an average of 9.2 entries in the symbol table for each function. The major considerations of the symbol table in **Lex** are not of any great importance in the symbol table for **Parse** because of the small number of symbols. Also, working with ids and not symbolic names means a much greater simplicity in the comparison of symbols. The entries in the chains are ordered by the value of the search key. The use of a single word as a search key makes the comparison relatively inexpensive and hence makes the symbol table search quite efficient.

A symbol table node, shown schematically in Figure 4.1, contains four fields. The **LINK** field contains a pointer to the next symbol node in the chain. The **REFERENCE** field contains a number which is unique to each node. The first symbol added to the table is given a reference number of 4 and each succeeding symbol is given a reference number which

is 2 greater than the last reference number assigned. All symbols are thus represented by even numbers, leaving the odd numbers to represent operators. The **NAME** field contains the Strings file id assigned to the symbol by **Lex**. The **TYPE** field contains a description of the nature of the symbol. The symbol can, for example, be a local variable, a label, or an external variable.

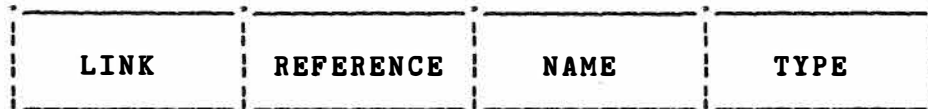


Figure 4.1 : Parse symbol table node

Since most of the time spent in the parsing phase is spent manipulating the parsing tables generated by YACC the symbol table described is quite sufficient. Data collected from tracing 12 compilations indicates that an average of 56% of the processor time used by **Parse** was spent manipulating the parsing tables while only .35% was spent manipulating the symbol table.

4.2 Expression Trees

An expression tree is built as an expression is parsed. These trees are made of 5 different types of nodes. Figures 4.2 through 4.6 show the formats of these nodes.

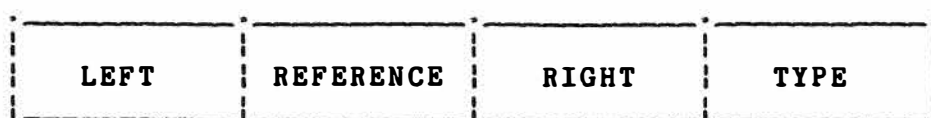


Figure 4.2 : Binary operator tree node

For binary operators the **LEFT** field contains a pointer to the sub-expression tree which is to be evaluated first. The **REFERENCE** field contains a number which identifies the binary operator. The **RIGHT** field contains a pointer to the sub-expression tree to be evaluated after the tree on the left. The **TYPE** field specifies whether the operator produces a name or a value when executed. This information, while available by identifying the operator, is best isolated for efficiency reasons. The type is needed since, for example, the "address of" operator can only be applied to a name and not a value. The check is not made syntactically in order to simplify the grammar. Also a more meaningful error message can be given when the error is detected during semantic processing.

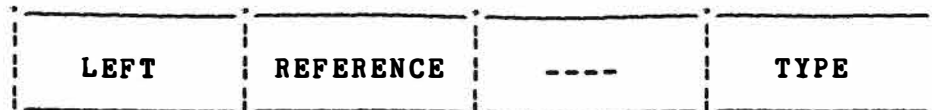


Figure 4.3 : Unary operator tree node

For unary operators the meanings of the fields are the same as for binary operators except that there is no right expression tree and the field reserved for that pointer is not used.

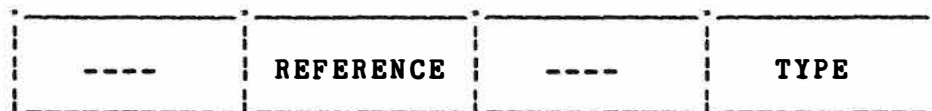


Figure 4.4 : Nullary operator tree node

For nullary operators neither the left nor the right expression trees exist and so both fields are unused. The **REFERENCE** and **TYPE** fields are similar to those for binary operators. Only one nullary operator exists at present. It is the operator which signifies the end of a list of arguments to be passed to a function.

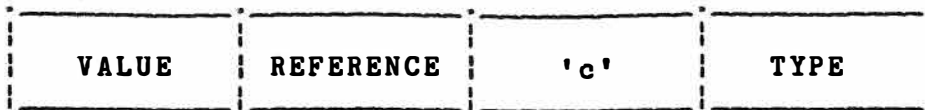


Figure 4.5 : Constant operand tree node

For constant operands all four fields are used. The **VALUE** field contains the value of the constant that the node represents. The **REFERENCE** field contains the reference number associated with the constant. Constants like symbols are uniquely identified by a reference number. The third field always contains the character 'c' to indicate that the node represents a constant. The **TYPE** field always specifies that the node is a value node rather than a name node. While this information is available by other means, the **TYPE** field is used so that there is only one method of determining the type for all nodes in an expression tree.

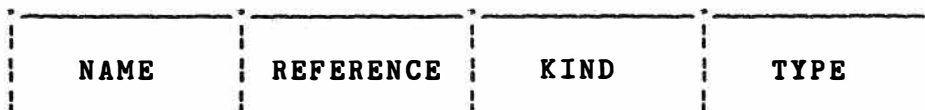


Figure 4.6 : Non-constant operand tree node

Non-constant operands also use all four fields in the node. The **NAME** field contains the Strings file id. The **REFERENCE** field contains the reference number associated with the operand. The **KIND** field indicates that the operand is a local variable, a label, a string, or an external variable. The **TYPE** field for strings and labels contains an indication that the node represents a value while all other operands are name nodes.

The form of an expression tree reflects both the order of evaluation required by the definition of Eh and the form of the expression as specified by the programmer. For operators whose right operand is evaluated first e.g. the assignment operators, the left sub-tree will represent the expression on the right of the operator as found in the source.

On output, the expression tree is walked in postorder and the reference numbers associated with the nodes are output. This produces a Polish-postfix representation of the expression tree, allowing all necessary information to be present while simplifying the task of rebuilding the expression tree in later phases. The tree is not simply output in full since the fields, other than the reference number, are not needed by later phases.

4.3 Constant Folding

Constant folding is done for the simple reason that it results in more efficient code and makes programming easier for the user. A programmer can write a constant as an expression of parts, thereby making the code more readable and maintainable. The precedence of operators is known by **Parse** and not **Lex**. Thus the folding cannot be done by **Lex**. Delaying folding to a later phase would cause increased complexity in the intermediate languages used by the compiler and complicate the detection of syntactically incorrect programs. Constant expressions that occur as executable expressions are no problem if folding is delayed however when the constants which are to be folded are used, for example, to specify the size of a vector the parser needs to know that the expression folds to a valid constant. Also the intermediate representation of the unfolded constant would have to be an expression thus complicating the language used to communicate between **Parse** and later phases.

It is assumed that both the host and target machines use twos-complement arithmetic. This could be changed by introducing small routines to perform each of the folding operations. Constants are folded with the precision of the

target machine. This is done for two reasons: so that **Code** will be able to create correct tables for select statements, and so that the values which result for the target machine are not dependent on the host machine.

Since the word length of the target machine is the only thing that would make **Parse** specific to a given machine it is passed a representation of this value on its command line. The representation chosen was to pass the value of the maximum positive number which would fit in a target machine word. From this number two other quantities are computed.

A destination word mask is created which is equivalent to ~ 0 on the target machine. A sign extended mask is also generated having a value which is equal to the host's ones complement of the target machines maximum integer.

When constants are read they are anded with the destination word mask to remove any excess precision present due to the word size of the host machine. If the resulting value anded with the sign extended mask is non-zero then the constant represents a negative value on the target machine and the value is ored with the sign extended mask to make it a negative constant on the host machine as well.

Constants are folded as an expression is parsed. This allows arbitrarily complex constant expressions to be used by a programmer with little chance that the space reserved for the expression tree will be exhausted.

The process of folding constants is actually more difficult than it appears. All constants to be folded have significant bits only within the bits covered by the destination word mask. After each operation is performed the result is anded with the destination word mask to remove excess precision and, if the sign bit is still set, the sign extended mask is ored into the result to preserve the sign.

For the operators +, -, *, &, |, ^, >, <, ==, !=, >=, <=, &&, and ||, the computation is simple and the operation is performed with the host machine hardware.

For division there are special cases. If the divisor is 0 then there is no simple way of predicting how the target machine will perform the "division". In this case **Parse** does not fold the constants but leaves this for the execution of the program on the target machine. A warning of the potential division by zero is given to the user. This is a warning, not an error message, since there is no means of knowing if the expression will be executed on the target machine, or if the programmer really intended to have it

done. An error message is only issued when the value of the expression is required to be a constant by the definition of Eh, say for the allocation of storage.

With division problems of portability also arise. Suppose that **Parse** is running on a Honeywell 66/60 compiling a program for a Honeywell 66/60. For obscure reasons the hardware will indicate a fault if the largest negative number is divided by 1. No other machine on which the compiler runs has this feature. Hence a special check is made for division by 1 since **Parse** should not be allowed to terminate abnormally due to user input. There are probably other machines with quirks as strange as this but as yet they have not been encountered.

The case of a right shift is also special. Since the shift operators of Eh are unsigned shifts, the value being shifted must first be anded with the destination word mask so that negative values after being shifted right by one or more will have 0's shifted into the high order bits of the word. This is necessary because of the possibility that the host word is larger than the target word. For example, a -1 held in a 36 bit word destined for a 16 bit word machine, shifted right by 2, still has the target machine's sign bit set. If the negative one is first anded with the destination

word mask then the correct result is obtained.

For a constant subscripted by another constant, the required actions are entirely different. Since a value indexed by another value is defined to be indirect through the sum of the two values the two constants are added together and the operator is changed to an indirection operator.

4.4 Error Recovery

Error recovery in a compiler tends to be a black art. The problem is very difficult. When a syntax error is detected the compiler is expected to continue parsing the input after the error is reported. Essentially the problem is: "What do you do when you do not know what to do?". One solution is to try to guess what the user might have done wrong. This works originally but eventually the user has been told about this problem so many times, what you have chosen as the probable cause will stop being the probable cause. YACC provides reasonable error recovery from syntax errors detected by the parser. Many errors are not detected by the parser, they are detected by action functions which check that certain conditions hold, for example checking

that a variable has been declared. These functions must, as well as detecting errors, leave the data structures in a condition that will not lead to "fallout" of non-errors generated by the first. A good example occurs in the function which checks that an expression is a constant. This function is expected to not only check for a constant but put the value of the constant in a specific place. If the expression was not a constant what value should be stored? If the value chosen is -1 and the constant was used to indicate the size of a vector then another error message would be generated objecting to a negative size. The value stored if an error is detected is highly specific to what the constant should be used for.

Parse attempts to overcome problems like this by carefully choosing actions on errors which will lead to the least chance of further complications. To achieve this a detailed examination of the actions taken on the recognition of each production in the grammar has been made. This is a "fine tuning" which has progressed throughout the life of the compiler and is likely to continue throughout the rest of its life.

4.5 Parsil

Parsil is the intermediate language used to convey the output of **Parse** to **Clean** and **Data**. It has declarations, expressions, conditional transfers, unconditional transfers, and labels. It also contains the same source location markers as **Lexil**. Since **Parsil** appears in two distinct files, one for function modules (the **Functions** file), and one for external modules (the **Externals** file), the grammar is described in two segments, after which certain interesting aspects of the language will be discussed.

One point must be made clear. **Parsil** is not a programming language for humans, it is a language for programs. **Parsil** has been designed for easy generation and interpretation, and not for human readability. Also, since it is the output language from a syntax checker, only syntactically correct programs are written in **Parsil**.

4.5.1 The Parsil Grammar

The grammar of **Parsil** (see Appendix B) is described in a manner similar to Backus-Naur form. The productions or rules are specified by placing the nonterminal being defined to the left of a "==" and the list of terminals and nonterminals which define it to the right of the "==". If there is more than one right-hand side for the same nonterminal, then the second and further right-hand sides will appear following the production, one per line, preceded by a "|". If there are no symbols to the right of a "==" or "|", then this implies that the nonterminal being specified can arise from null input. It is not the intention of this section to go deeply into the grammar of **Parsil**, rather some aspects of the language will be discussed.

Parsil is a statement oriented language. Each statement type is uniquely identified by the first character of the statement. Most statements are of known length thus simplifying their manipulation. Since different statements appear for data and functions they will be examined and discussed separately in the next two sections.

4.5.2 The Externals File

Parsil statements found in the Externals file convey the external module declarations found in the source. To a person familiar with the grammar of Eh the correspondence with the source should be obvious in most cases.

There are four statements which can appear in the externals file: the simple external, string external, and vector external declarations, and the end of input statement. These four statements are identified by the initial characters ':', '"', '[', and '*0' respectively.

Of these four, the end of input statement is the simplest. When a null byte is encountered there is no more input.

Next in simplicity is the string declaration. The string declaration consists of the Strings file id of the name of the external, the minimum size of string, and the Strings file id of the literal string with which the external is initialized.

For the simple external there is no size information. There is only the Strings file id of the name and an initializer. The initializer may be either a constant or a

name. If the initializer is a name it is specified by the character 'g' followed by the Strings file id of the name of the identifier. The initializer being a constant is detectable since there will be no 'g' to start the initializer. Note that 'g' has a value which is less than 127. The constant is represented in exactly the same form as it was in Lexil.

The vector external is by far the most complex. Like the string external, the Strings file id of the name of the external and the minimum size are specified. However there is a list of initializers. Each of these initializers can be either a constant, a string, a name, or a pointer to another list of initializers. These initializers are distinguished by the first character of the representation. An initial 'g' is used to specify that the initializer is a name as with the simple external. If the initializer is a string then it begins with a '"' followed by the Strings file id of the literal string. A pointer initializer is specified by a simple '['. A ']' is used to signify the end of the list of initializers of the sub-vector. As with the simple external, constants are represented by the same means as they are in Lexil. Due to the fact that there can be an arbitrary number of initializers, the vector external has to have a terminator. A null byte is used for the terminator since it

cannot be confused with a valid initializer.

4.5.3 Functions File

Within the Functions file there are two levels of **Parsil**. At the outer level there are statements which announce either a function, an external, or the end of input. The external declaration is there only to inform **Clean** of the definition of a data module by the user. How the external declaration is used will be seen when **Clean** is discussed. The function statement denotes the beginning of a function module. When one of these statements is encountered, a switch is made to the second level. The end of input statement, as its name implies, marks the end of input. There is one other type of statement which can occur outside a function module, the source location marker statement. These markers can occur between any two statements in the Functions file.

The only declarations that appear before the body of the function are the declarations for arguments, auto variables, and externals referenced in an **extrn** statement in the source. External declarations can appear within the body of a function for calls to another function. Constant and

string declarations also appear within the body of the function.

There is only one type of conditional transfer, the jump-on-false statement. All conditional transfers are forward transfers.

4.5.4 Points of Interest

One of the more interesting aspects of Parsil is the terminal called REFERENCE. This is the reference number mentioned earlier. The major problem with reference numbers was finding a reasonable means of representation. Two-byte representations were tried but it was noticed that there were many bytes in the representation which were null, due to the fact that there are only a small number of operators and that a typical function had a small number of variables, constants, and labels. One-byte representations were dismissed since they would severely limit the size of function that could be represented, or require a complicated set of rules governing the redefinition of reference numbers.

The solution chosen was to use a variable number of bytes in the representation. The number of one-byte representations is reduced from 256 to 128. If the value of the first byte of a **REFERENCE** is greater than 127 then the representation takes two bytes. The second byte, if needed, follows the first. By simply not using a byte with a value of 0 to represent any operator or operand, 0 can be used to terminate lists of reference numbers.

Another important aspect of **Parsil** is that it is easily extended. For example, **Parsil** has been made to function for a typed language simply by adding type information to the declaration statements.

Chapter 5

Function Modification

Parse generates an intermediate version of the source code which may contain unreachable statements and transfers to labels that are followed immediately by transfers to other labels. It does so because it does not have an overall view of the function being compiled. **Clean** has an overall view and can produce a better version by modifying or removing unreachable statements, transfers to transfers, redundant labels, etc.

Clean also modifies expressions to help the code generator produce better code. Many modifications can be made which are not influenced by the target machine.

Checks are also made to be sure that the correct number of arguments are passed to called functions. Function calls are modified so that functions that do not need to know the number of arguments they are called with, are not passed this information. This modification saves more space in the final executable program than most other modifications, as well as lowering the time required to call many functions.

5.1 Structure

Clean executes in three subphases. The first subphase reads, modifies, and outputs the functions which constitute the source being compiled. These operations are performed on a function by function basis. The number of formal arguments the function was defined with is recorded. As a function is read, declarations are immediately output but all other statements are stored in memory.

The structure of the stored function is modified repeatedly until no more modifications are possible. The expressions are changed to remove unnecessary sub-expressions and to simplify the task of the code generator. If it is discovered that the function needs to know how many arguments it is called with, this information is recorded. If the function returns a value this is also recorded.

The fully modified function is written to the output file. All calls made by the function are recorded for use in the third subphase of **Clean**.

If any functions were called that were not supplied in the source, then the second subphase of **Clean** is invoked. The second subphase will search any libraries which were specified, looking for the definitions of the missing functions.

The third subphase of **Clean** checks that every function call was made with a correct number of arguments. Calls to functions which do not need to know about the number of arguments passed to them are modified so that **Code** will generate the simpler call. The definition of the called function is also checked to see if it returns a value, should the caller expect a value to be returned.

5.2 Input

Input proceeds in two distinct modes. In the first mode definitions of modules are processed. The second mode is concerned with the internal statements of a function. In both modes, source location markers may be encountered. These markers will be ignored in future discussions.

5.2.1 Module Definition

In the first mode of input there are three valid characters which may be read: ':', '(', and the null character. Any other character encountered is an error and compilation is terminated.

When a ':' appears it is followed by the identification of an external variable. External declarations are passed to **Clean** for two reasons. The first, and most important, is that it defines a module which is not a function so that, should the programmer refer to this module as a function, a warning can be given. The second reason is the detection of multiple definitions. While multiple definitions will eventually be caught by the loader, it is better to have them found as early as possible.

If a null character is encountered, all of the input has been read. **Clean** may then proceed to the second subphase, the searching of libraries.

If a '(' is read, a function has been encountered. This is the signal for input to switch to the second mode.

5.2.2 Function Input

Declaration statements are not kept in memory but are passed to the output file. The only pieces of information retained are the values of constants.

Each statement is represented in memory by a simple data structure. A vector is allocated for each statement: one word will point to the next statement, the rest of the vector holds the statement.

When a null character is read from the input, the function is assumed to be terminated, and control passes to the functions which modify the loaded function.

5.3 Structure Modification

The modification of the structure of a function is performed in seven separate sections. Each section is executed in turn until all have been done. If any of the sections manage to change anything, then all the sections are executed again. These sections are described in detail later so only a brief summary will be given here.

Unreachable statements, statements which follow a transfer or a return statement up to but not including the next label found, are deleted. Any labels which are not referenced are removed. Null transfers, a transfer to a label which immediately follows the transfer, are deleted. Any expression which is used to control a conditional transfer to a label, which is followed immediately by an unconditional transfer to the same label, is changed to an expression which does not control a transfer since the transfer will take place in any event. Any expression which has no effect, e.g. no possible modification of a memory location, is removed. Any transfer to a transfer is changed to a transfer to the target of the second transfer. This is termed straightening transfers. If a conditional transfer over a transfer exists, then the conditional transfer is changed to transfer on the reverse condition, the label of the conditional transfer is changed to the label of the following unconditional transfer, and the unconditional transfer which followed is removed.

Most modifications, with the exception of removing unreachable code and straightening transfers, do very little in the way of improving the function either in size or speed of execution. What they do do, however, is simplify the function so that more unreachable code, and transfers to

transfers, can be detected.

These modifications are done only if no programmer-supplied machine instructions (twits) are present. These instructions can effect the flow of control in ways that **Clean** can not determine. When the programmer attempts something that the compiler does not understand, it is better to leave certain inefficiencies than to defeat the programmer. The one most important criterion when **Clean** was designed and implemented was that it should only improve a program, not change it to the point that it becomes a different program.

5.3.1 Unreachable Statements

Unreachable statements can be produced by **Parse**; it ignores efficiency aspects which are the province of **Clean**. An example of this can be found in the function **Get_ref** which reads a reference number from the **Parsil** file. The source of **Get_ref** follows:

```

Get_ref()
{
    auto c;

    if( (c=.Get()) & $80 ) return( c&$7F | (.Get())<<7 );
    else return( c );
}

```

The if ... else ... construct comes from Parse, essentially, as the following.

```

    if( !((c=.Get()) & $80) ) transfer label10;
    return( c&$7F | (.Get())<<7 );
    transfer label16
label10:
    return( c );
label16:

```

The "transfer label16" is unreachable because of the return and it is removed by Clean.

Since the statements which are removed are unreachable, their removal will not effect the execution of the function.

5.3.2 Straighten Transfers

One of the common inefficiencies in the output of a parser is the existence of transfer chains. A transfer to a transfer is very often generated by `Parse`. The changing of the target label of the original transfer to the label of the last transfer in the chain can lead to faster execution times.

A function which displays transfer chains is `Skip_blanks_etc.`, a function in `Lex` which is used to skip over any blanks, tabs, or form feeds in the source. The source is:

```
Skip_blanks_etc.()
{
    auto c;

    repeat
    {
        c= Get_ind();
        if( c != ' ' && c != '\t' && c != '\f' ) break;
    }
    Backup();
}
```

From `Parse` this is the representation `Clean` has to work with:

```

label4:
    c=Get_ind();
    if( !(c!=' ' && c!='t' && c!='f') ) transfer label20;
    transfer label6;
label20:
    transfer label4;
label6:
    Backup();

```

Any transfer to label20 is really a transfer to label4. After the transfer has been modified the function will look like the following:

```

label4:
    c=Get_ind();
    if( !(c!=' ' && c!='t' && c!='f') ) transfer label4;
    transfer label6;
label20:
    transfer label4;
label6:
    Backup();

```

Skip_blanks_etc. will be used in later sections to show the effect of further modifications. Simply straightening the one transfer is not all that can be done to it.

5.3.3 Unreferenced Labels

Continuing with the example used in the last section, after the transfer was straightened the function looked like:

```
label4:
    c=Get_ind();
    if( !(c!=' ' && c!='\t' && c!='\f') ) transfer label4;
    transfer label6;
label20:
    transfer label4;
label6:
    Backup();
```

Label20 became unreferenced and so can be removed.

Removal of labels, in itself not saving anything in final executable size, simply makes it easier to detect other unreachable statements. In this example the transfer to label4 which followed the definition of label20 is now unreachable and can be removed resulting in a version of the function which looks like:

```

label4:
    c=Get_ind();
    if( !(c!=' ' && c!='t' && c!='f') ) transfer label4;
    transfer label6;
label6:
    Backup();

```

5.3.4 Null Transfers

A null transfer, a transfer to a label which immediately follows the the transfer, usually occurs because of other modifications already made.

A function which displays a null transfer is **Skip_blanks_etc.** as it exists after unreachable statements are removed, transfers have been straightened, and unreferenced labels have been deleted. As seen in the previous section, there is a transfer to **label6** immediately followed by definition of **label6**. The transfer to **label6** is simply removed. Note that now **label6** is an unreferenced label and it can be removed. Now the function has been reduced to the following.


```

label4:
    c=Get_ind();
    if( !(c!=' ' && c!='*t' && c!='*f') ) transfer label4;
    Backup();

```

The removal of null transfers save little in final size of the compiled program, but their removal lowers the reference count on a label by one. In this example there is no more unreachable code however, in many functions, more unreachable code could possibly be made detectable to **Clean** by removing null transfers.

5.3.5 Conditional to Expression

A conditional statement is of the form: "if expression transfer label". Occasionally a conditional statement will be followed by an unconditional transfer to the same label. In this event there is no need to conditionally transfer to the label since the transfer of control is inevitable. Thus the pair of statements:

```

    if expression transfer labelA
    transfer labelA

```

become the two statements:

expression
transfer labelA

On many machines conditional transfer instructions have smaller ranges than unconditional instructions (because of the extra bits in the instruction used to encode the condition). The PDP/11 is a good example: conditional transfers have a limited range of destination, unconditional transfers can be made to any addressable location. Because of the limited range, a conditional transfer might have to be made to a location containing an unconditional transfer, just to achieve addressability.

While removing the conditional jump can lead to a saving in space its main advantage is that it opens another statement to modification by the function which searches for useless expressions which, as shall be seen later, can lead to more improvements.

Since the flow of control is not modified, and the expression will still be evaluated, this modification will not change the meaning of the function.

5.3.6 Remove Useless Expressions

A useless expression is an expression which contains no function calls and which changes no addressable locations. Since a useless expression has no effect it can be removed without changing the meaning of the function.

These useless expressions are usually created when a conditional expression is changed to a simple expression. Any conditional statement which was changed into a simple expression statement thus becomes a candidate for removal.

5.3.7 Reverse Conditional Transfers

If a conditional transfer has as its target the label immediately after the next statement, and that next statement is an unconditional transfer, then the conditional transfer is changed to a transfer on the reverse condition, and the target is made the target of the following unconditional transfer. The unconditional transfer is then removed.

What this means is that the construct,

```
if !expression transfer label1
transfer label2
label1
```

becomes

```
if expression transfer label2
label1
```

For example, there is a function in `Lex` which handles the `@` command, called `Set_at`. One of the things it does is remove leading blanks and tabs found before the pathname. The statement used is:

```
while( (c=.Get())==' ' || c=='*t' );
```

Input to `Clean`, it looks like this.

```
label32:
    if( !((c=.Get())==' ' || c=='*t') ) transfer label34;
    transfer label32;
label34:
```

The transfer to `label34` can be changed to a transfer to `label32`, the conditional test can be reversed, and the transfer to `label32` can be removed. After the operations are performed this is what results. (`label34` will be removed if no other references to it are found)

```
label32:
    if( ((c=.Get())==' ' || c=='*t') ) transfer label32;
label34:
```

5.4 Expression Modification

Expressions are modified for two reasons. Some modifications simplify the expressions, creating smaller executable functions. Others make the job of code generation easier by adding information to the expression trees which the code generator can use.

There are five basic modifications which are done. A Sethi-Ullman algorithm is performed on the expression tree, placing, where possible, the more complex operand on the left of the operator. If an operator has a constant as its left operand the operands are interchanged and the operator changed to reflect this fact. If a subscripting operator represents a value rather than an address, then another operator is inserted above the subscripting operator to tell the code generator that the address is not required. Pairs of operators which can be represented by a single operator are changed to the single operator. Certain expressions, which have as one operand a constant with a value of 0, are modified, either by replacing the expression with the constant 0 operand, or by removing the operator at the top of the expression tree along with the constant 0 operand. These modifications will be discussed in more detail below. Calling a function or storing a value into an addressable

cell is termed an effect. Some of these modifications can only be performed if the sub-expression being modified has no effects.

Throughout this section it might appear that a great deal of attention has been given to the subscripting operators. Static profiles of Eh functions show that the subscripting operators are some of the most commonly used operators. As such, these operators are critical to the size of the resulting compiled function.

5.4.1 Sethi-Ullman Balance

Each expression tree is walked and the sub-trees are swapped if it is discovered that the right sub-tree requires more temporaries than the left sub-tree. This algorithm is known as a Sethi-Ullman balance. Swapping of sub-trees can be applied only when the two sub-trees have no effects.

Since this is a modification common to many compilers no more will be said here. The interested reader will find a detailed description of this algorithm in Sethi, (1970).

5.4.2 Swapped Operators

Many operands found in expressions are constants. Constants are different from other operands for the simple reason that their values cannot be affected by any operation.

It thus becomes advantageous to evaluate the other, more complicated, operand of a binary operator before evaluating the constant. For operators which are commutative, for example addition, the sub-trees can be swapped and there is no need to inform the code generator that the swapping has been performed. For relational operators, simply changing the operator to the reverse-sense operator is sufficient, e.g. `constant < operand` changes to `operand > constant`.

There are many operators which either are not commutative, or do not have reverse operators in the source language. Here the concept of a swapped operator is used. For example, in the expression,

A = 3

Clean is given a left operand of 3 and a right operand of **A**, with an assignment operator. When the sub-trees are swapped, the operator is changed to a "swapped assignment". The swapped assignment will assign the value of its right

operand to the cell named by its left operand.

Knowledge that the right operand is a constant, which the code generator can discover from the operator, can vastly simplify its task of generating machine instructions.

5.4.3 Fetch Operators

A subscripting operator can either represent a name or a value depending on context. Whether a value is required or not can be discovered by **Code** when the time comes to generate code, however **Code** can be simplified if it can be told this explicitly. **Clean** inserts fetch operators over all subscripting operators which are used to access values. Figure 5.1 shows a subscript operator which is used to retrieve a value, and the resulting expression when a fetch operator is inserted. Because there are two subscript operators, one for words and one for bytes, there are two load operators, a word fetch, and a byte fetch.

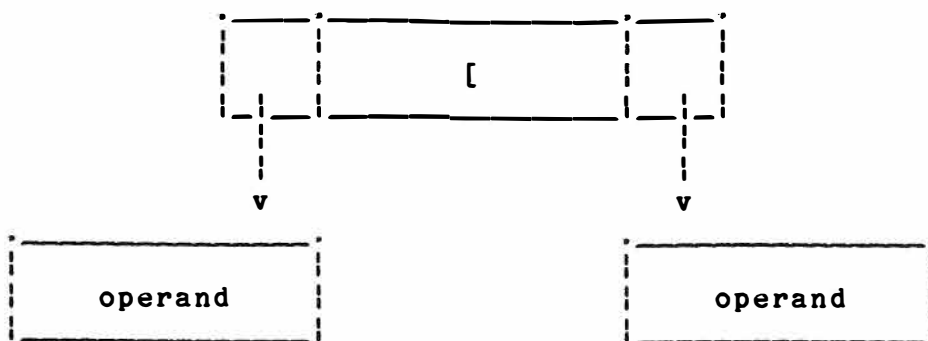


Figure 5.1a: Subscript expression for value

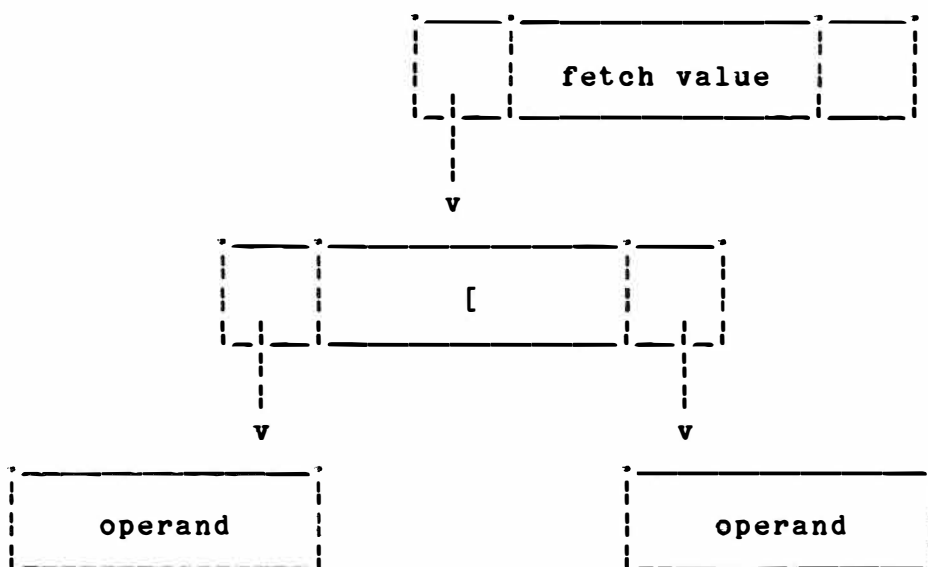


Figure 5.1b: Fetch operator added to subscript

5.4.4 Modify Operators

Certain pairs of operators found in expressions can be simplified. By the definition of Eh, the expression, $\&\#t$ is equivalent to t and this is one of the simplifications possible. This is shown in figure 5.2.

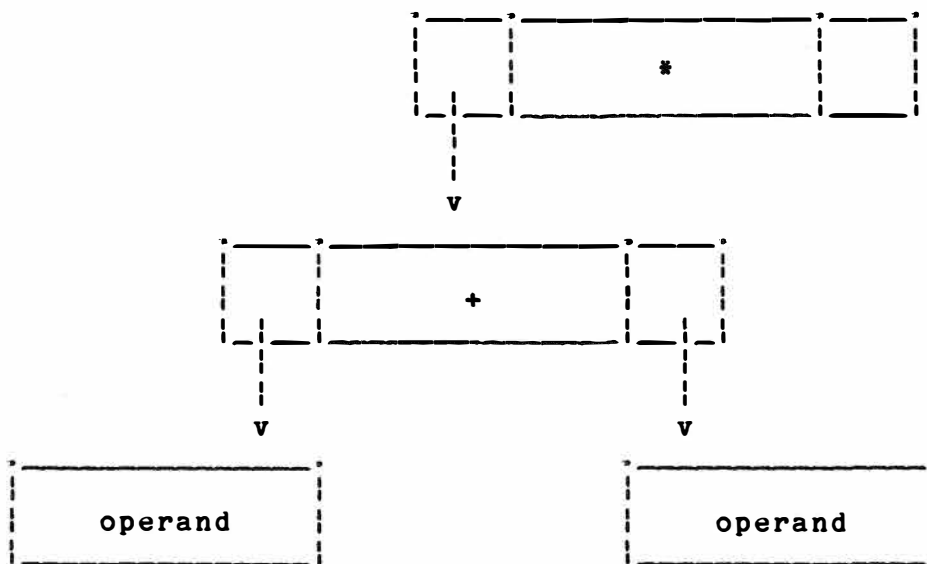


Figure 5.2a: Indirect of plus expression

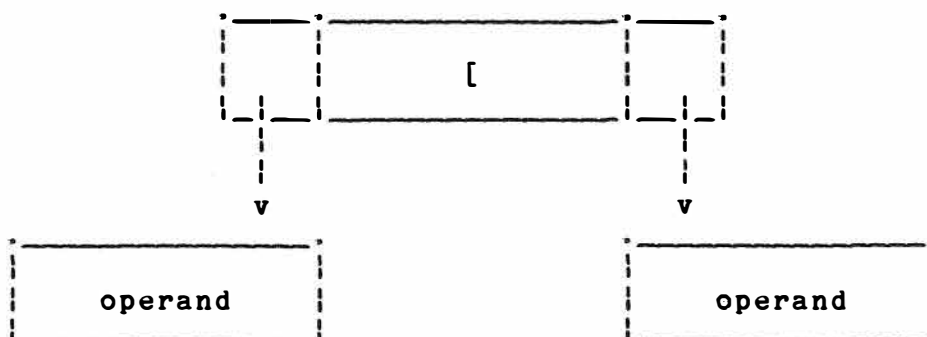


Figure 5.2b: Indirect of plus changed to subscript

Because the word subscripting operator is defined by the relation, $a[b] == *(a+b)$ this allows the expression $\&a[b]$ to be changed to $a+b$ as shown in figure 5.3.

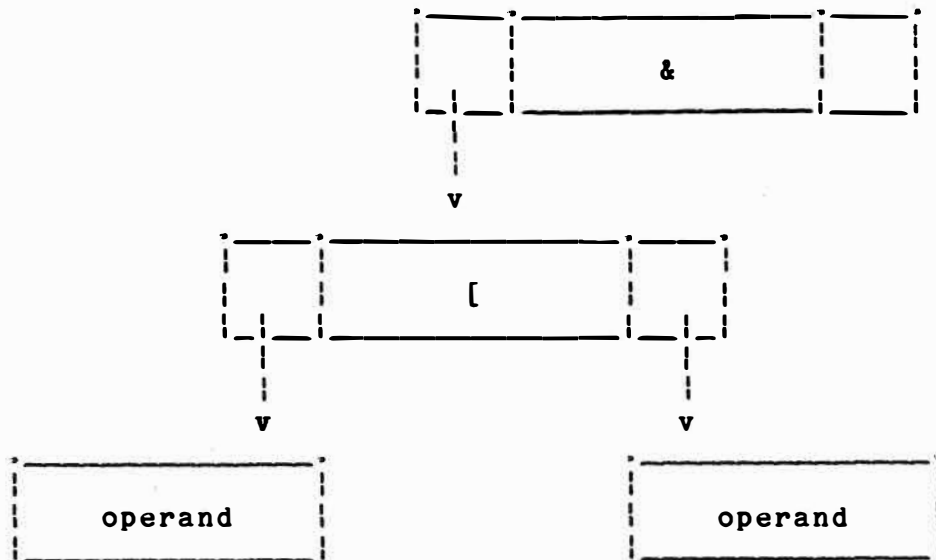


Figure 5.3a: Address of subscript expression

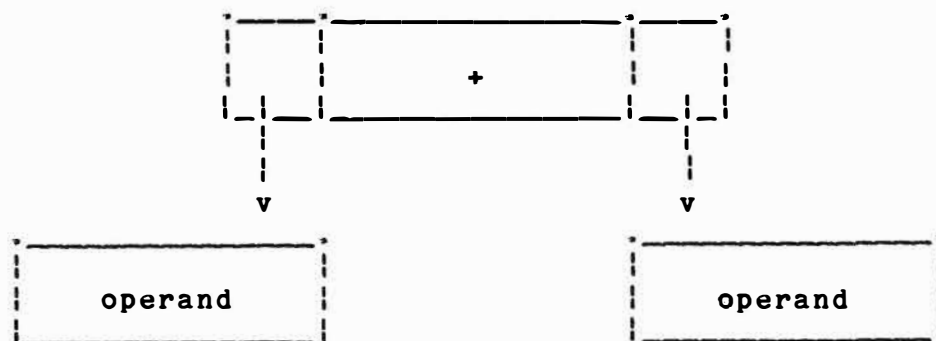


Figure 5.3b: Address of a subscript modified to a plus

5.4.5 Remove Useless Operands

A useless operand is one for which the result of its evaluation is known at compile time. Many expressions, if one of the operands is a constant with a value of 0, can be simplified. Some can be simply modified from an expression, to a constant with a value of 0, others get changed to their non-zero operand.

There are 6 cases which must be examined. For these examples, the symbol `operand` will represent any valid sub-expression.

Case 1

If the expression is any of the following:

```

operand % 0
operand + 0
operand - 0
operand >> 0
operand << 0
operand ! 0
operand ^ 0
operand %= 0
operand += 0
operand -= 0
operand >>= 0
operand <<= 0
operand != 0
operand ^= 0

```

it can be seen that the result of the evaluation would simply be the value of `operand`. Even for the assignment

operators, there will be no change in any addressable locations. This modification is shown graphically in Figure 5.4.

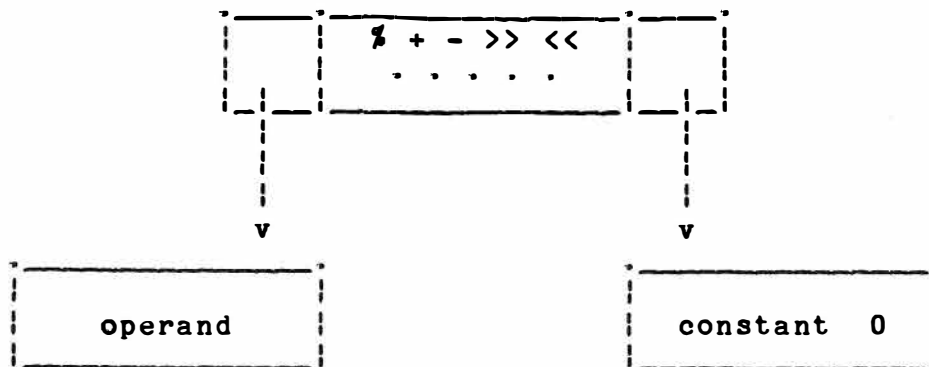


Figure 5.4a: Expression with useless constant 0 operand



Figure 5.4b: Expression replaced by left operand

Case 2

Some operators would, rather than producing the evaluation of the non-zero operand, produce a 0 instead. The expressions which would give rise to these conditions are:

```

operand * 0
0 / operand
0 % operand
operand & 0
operand && 0
  
```

Care must be taken however, since simply removing the

operator and non-zero operand might have undesirable effects. If the operand has any effects then the expression, instead of being simply modified, would be changed. Only if the non-zero operand has no effects can the modification shown in Figure 5.5 be made.

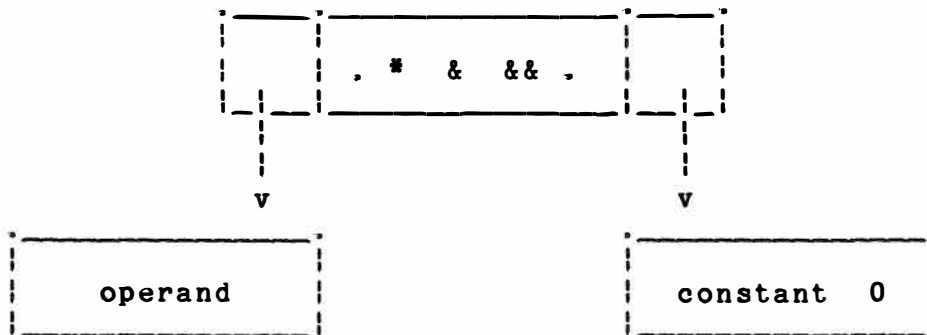


Figure 5.5a: Expression with useless non-zero operand

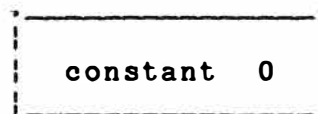


Figure 5.5b: Expression replaced by constant 0

Case 3

If the operator is a division operator, no modifications are made, however the user is warned of the division by 0.

Case 4

If the expression is,

`operand[0]`

Then it is changed to

***operand**

since the definition of subscripting is indirect through the sum of the two operands.

Case 5

If the expression is

0 - operand

then the operator gets changed into a unary negation, rather than a subtraction.

Case 6

Finally, if the expression is

operand *= 0

it is changed into

operand = 0

since there is no need to multiply by 0 to set the location represented by **operand** to 0.

5.5 Output

After the function has been modified it is placed in the output file. Since the declarations are copied when the function is loaded only the body of the function has to be output.

As the expressions are copied, the Strings file ids of functions called and the number of arguments passed to them are recorded. The recording is done in a work file. Figure 5.6 shows the format of the record of the function call.

WHERE	WHO	NUM	VAL	LINE NUM	FILE NAME
-------	-----	-----	-----	----------	-----------

Figure 5.6 : Function call record

The **WHERE** field, two bytes in length, records where in the output file the call has been placed. This information will be possibly used later, to change the type of function call to one which does not pass an indication of the number of arguments.

The **WHO** field, two bytes, records the Strings file id of the function called. This will be used for associating function calls with definitions.

The **NUM** field, one byte in length, contains the number of arguments the called function is passed. This will be used to check that the correct number of arguments have been passed.

The **VAL** field, one byte, contains a logical value, **TRUE** if the function is expected to return a value, **FALSE** if it is not.

The **LINE NUM** field, two bytes, contains the line number in the source file where the call was made.

the **FILE NAME** field, two bytes, contains the Strings file id of the file in which the call was made. This field, along with **LINE NUM**, is useful if a warning message has to be given concerning the call.

The work file will be processed in the third subphase of **Clean**, maintaining the information needed to verify that all function calls conform to the expectations of the function being called, and that the function called conforms to the expectations of the function performing the call.

5.6 Library Searching

Library searching is done after the entire program being compiled has been processed. Usually there are some functions called that were not included in the source. In order to know if these functions were called correctly, **Clean** reads through the libraries looking for the definitions of the missing functions. The libraries are searched in the appropriate order until all undefined functions have been found.

Each function in the library has certain information recorded about it, in particular, the minimum number of arguments, maximum number of arguments, whether it needs to know how many arguments it is called with, and whether or not it returns a value.

The functions not included in the source are found in these libraries, if they exist, and the needed information about them is recorded. This information, with that built up during the first subphase of **Clean**, allows the third subphase of **Clean** to check all the function calls and simplify those calls that do not need the number of arguments passed.

5.7 Function Call Checks and Modification

As the file containing the records of the calls made is read, each call is checked against the definition record built up either during the processing of the input, or the searching of the libraries. If the call does not correspond with the definition, a warning message is output.

After the call record is used to check function calls, the type of call needed is checked. If the function being called does not need to know how many arguments it is called with, the information in the **WHERE** field of the call record is used to seek to, in the output file, the location where the call is made. Since the encoding for both types of function call takes one byte of space the encoded call is "patched" to the simpler call if possible. Because the calls are recorded in the order they are encountered in the output file, the seeking is always toward the end of the file. The output files is thus processed sequentially in large steps.

5.8 Cleanil

Cleanil is the language used to communicate the program being compiled to **Code**. On the surface it is almost identical to **Parsil**. There are, however, some rather important differences. The grammar of **Cleanil** is given in Appendix D, in the same format as the grammar of **Parsil**.

As well as a conditional transfer on false, there is a conditional transfer on true. The modifications performed by **Clean** can create a conditional jump on true by reversing the conditional jump in order to simplify the function.

Except for labels, all definitions are made before the body of the function is entered. Labels are defined at the locations they refer to.

Chapter 6

Conclusions

Structuring the Eh compiler as a set of phases allows the compiler to run on minicomputers. Although it executes at a slower rate than one large monolithic phase, other benefits are gained. One benefit is that better code can be generated for compiled programs. The ability of a phase to process all of the source before any code is generated, as **Clean** has, allows global optimizations on function calls that are not possible without excessive requirements on the user.

Removing the lexical scanning from the parsing has simplified maintaining both phases. Because interactions between these two sections of the compiler have been limited to two files, modifications can be made to one of the phases, and as long as the interface is consistent, there are no changes needed to the other. This separation has, however, produced a rather long delay before any of the source being compiled is seen by the parser.

Allowing **Clean** to make many passes over the function being compiled gives it the ability to find modifications that are possible due to other modifications that it has performed. The removal of unreferenced labels exposing more unreachable code is one example. The ability to keep a representation of the entire function in memory allows multiple passes to be done efficiently, but this ability would not be present if there was code present in the phase for parsing or code generation, due to size requirements. Keeping the representation in memory, while efficient, does limit the size of function which may be compiled.

The checking of function calls that **Clean** does exposes some of the most obscure errors in programs. Without this checking, some programs can "work" for suprisingly long periods of time, until a minor modification causes the whole program to start acting "strangely".

6.1 Future Work

The assumption made by **Parse** and **Clean** about the host word size being at least as big as the target word size must eventually be removed. This can be done for **Parse** by creating a set of multiple precision arithmetic routines. It is not yet known what effect this extra processing will have on the performance of **Parse**, however it is expected to be slight due to the fact that the manipulation of the parsing tables seems to be the major use of time during parsing. For **Clean**, supporting multiple precision is simple since it does no arithmetic with constants, but only checks for certain values.

Loading the entire body of a function into memory as **Clean** does now, puts a restriction on the size of function which can be compiled. This limit is approximately 2000 statements. If an additional statement is added to **Clean1** which is an ignore statement, then **Clean** could possibly work with a much smaller in-core version of the function, increasing the limit to 4000 to 6000 statements. This ignore statement could be written over the output of **Clean** to remove the statements that it had found to be not necessary. All that would have to be in memory for each statement would then be An indication of the statement type and the location

in the output where the statement is found.

An improvement could be realized in the final code generated if **Clean** were to collect all expressions between labels and transfers, into one large expression. The total saving in final size should be investigated to see if this is a viable modification. It may place too big a load on the memory requirements of **Code**, however.

At present the compiler executes as a sequence of phases but there is no reason all phases except **Code** could not execute at the same time. The reason that **Code** could not be executed at the same time as the other phases, is that **Clean** must make two passes over its output, one to create it, and one to modify it. Creating another concurrent team which would take function definition information from **Parse** or **Clean**, and give this information to **Code** as needed, might make the restriction go away. Running the compiler as a set of concurrent teams instead of as a sequence might have a beneficial effect on the time needed to compile a program, however the effect is not known.

Bibliography

- Braga, R.S.C.(1976), "Eh Reference Manual", Research report, CS-76-45, University of Waterloo Dept. of Computer Science, November.
- Braga, R.S.C., M.A. Malcolm, G.R. Sager(1976), "A Design for a Portable Programming System", Research report, CS-75-29, University of Waterloo Dept. of Computer Science, November.
- Richards, M.(1969), "BCPL: a Tool for Compiler Writing and System Programming", Proc. Spring Joint Computer Conference
- Knuth, D.E.(1973), "The Art of Computer Programming Vol. 3", Addison-Wesley, Reading, Massachusetts.
- Cheriton, D.R., M.A. Malcolm, L.S. Melen, and G.R. Sager(1978), "Thoth, a Portable Real-time Operating System (revised)", Research report, CS-78-15, University of Waterloo Dept. of Computer Science, March, to appear in Comm. A.C.M.
- Johnson, S.C. and B.W. Kernighan(1973), "The Programming Language B", Bell Laboratories Computer Science Technical Report, No. 8, January.
- Johnson, S.C.(1974), "YACC - Yet Another Compiler Compiler", Bell Laboratories Computer Science Technical Report, No. 32.
- Wulf, W., R.K. Johnson, C.B. Weinstock, S.O. Hobbs and C.M. Geschke(1975), "The Design of an Optimizing Compiler", American Elsevier, New York.
- Bonkowski, G.B.(1978), "Structure of the Eh Compiler: Code Generation", Masters Thesis, University of Waterloo Dept. of Computer Science, September.
- Sager, G.R.(1977), "The Thoth Linking Loader", Research report, CS-77-15, University of Waterloo Dept. of Computer Science, January.

Sethi, R. and Ullman, J.D.(1970), "The generation of optimal code for arithmetic expressions", JACM 17 October.

A. Lexical Scanner

Following is the source for the main function which performs token analysis in Lex:

```
Lex()
{
    extrn Line_number, In_pushed_back;
    auto token, char;

    repeat
    {
        select( char=Get_ind() )
        {
            case ' ' :      \ ignore blanks
            case '*t' :      \ and tabs
            case '*f' :      \ and form feeds
                next;

            case 0 : return;    \ end of input. finished.

            case '*n' :
            {
                token = END_LINE;
                ++Line_number;
            }

            case '#' :      token = NUMBER_SIGN;

            case '(' :      token = OPEN_PARENTHESIS;

            case '[' :      token = OPEN_BRACKET;

            case '{' :      token = OPEN_BRACE;

            case ',' :      token = COMMA;

            case '~' :      token = ONES_COMPLEMENT;

            case '?' :      token = QUERY;

            case ']' :      token = CLOSE_BRACKET;
```

```

case ')' :      token = CLOSE_PARENTHESIS;
case '}' :      token = CLOSE_BRACE;
case ';' :      token = SEMI_COLON;
case '/' :
    if( (char=Get_ind()) == '=' )
        token = ASSIGN_DIV;
    else
    {
        token = DIVIDE;
        In_pushed_back = TRUE;
    }
case '\\' :
    {
        while( ( char=Get_ind()) != '*n' && char );

        In_pushed_back = TRUE;
        next;
    }
case '!' :
    if( (char=Get_ind()) == '=' )
        token = NOT_EQUAL;
    else
    {
        token = NOT;
        In_pushed_back = TRUE;
    }
case '+' :
    if( (char=Get_ind()) == '+' )
        token = INCREMENT;

    else if( char == '=' ) token = ASSIGN_ADD;

    else
    {
        token = PLUS;
        In_pushed_back = TRUE;
    }

```

```

    }
case '-' :
    if( (char=Get_ind()) == '-' )
        token = DECREMENT;

    else if( char == '=' ) token = ASSIGN_SUB;

    else
    {
        token = MINUS;
        In_pushed_back = TRUE;
    }
case ':' :
    if( (char=Get_ind()) == ':' )
        token = COLON_COLON;

    else
    {
        token = COLON;
        In_pushed_back = TRUE;
    }
case '|' :
    if( (char=Get_ind()) == '|' )
        token = LOGICAL_OR;

    else if( char == '=' ) token = ASSIGN_OR;

    else
    {
        token = BITWISE_OR;
        In_pushed_back = TRUE;
    }
case '^' :
    if( (char=Get_ind()) == '=' )
        token = ASSIGN_XOR;

    else
    {
        token = EXCLUSIVE_OR;
        In_pushed_back = TRUE;
    }

```

```

    }

case '&' :

    if( (char=Get_ind()) == '&' )
        token = LOGICAL_AND;

    else if( char == '=' ) token = ASSIGN_AND;

    else
    {
        token = AMPERSAND;
        In_pushed_back = TRUE;
    }

case '**' :

    if( (char=Get_ind()) == '=' )
        token = ASSIGN_MULT;

    else
    {
        token = ASTERISK;
        In_pushed_back = TRUE;
    }

case '%' :

    if( (char=Get_ind()) == '=' )
        token = ASSIGN_MOD;

    else
    {
        token = MODULO;
        In_pushed_back = TRUE;
    }

case '<' :

    if( (char=Get_ind()) == '<' )
        if( (char=Get_ind()) == '=' )
            token = ASSIGN_LEFT_SHIFT;

    else
    {
        token = LEFT_SHIFT;
        In_pushed_back = TRUE;
    }

```

```

else if( char == '=' )
    token = LESS_OR_EQUAL;

else
{
    token = LESS_THAN;
    In_pushed_back = TRUE;
}

case '>' :

    if( (char=Get_ind()) == '>' )
        if( (char=Get_ind()) == '=' )
            token = ASSIGN_RIGHT_SHIFT;

        else
        {
            token = RIGHT_SHIFT;
            In_pushed_back = TRUE;
        }
    else if( char == '=' )
        token = GREATER_OR_EQUAL;

    else
    {
        token = GREATER_THAN;
        In_pushed_back = TRUE;
    }

case '=' :

    if( (char=Get_ind()) == '=' ) token = EQUAL;

    else
    {
        token = ASSIGN;
        In_pushed_back = TRUE;
    }

case '1' :: '9' :
{
    In_pushed_back = TRUE;
    Process_const( 10 );
    next;
}

case '0' :
{

```

```

        Process_const( 010 );
        next;
    }

case '$' :
{
    Process_const( $10 );
    next;
}

case 'a' :: 'z' :
case 'A' :: 'Z' :
case '_' :
case ',' :
{
    In_pushed_back = TRUE;
    Process_ident_or_kw();
    next;
}

case "'" :
{
    Process_char();
    next;
}

case '"' :
{
    Process_string();
    next;
}

default :
{
    Error( "bad character in source" );
    next;
}
}
.Put( token );
}

```


B. Parsil Grammar

Because **Parsil** exists both for functions and for externals, the grammar is presented in two sections. Tokens are shown either as quoted characters, or in upper case.

External Parsil

```
file           := module_list '*0'
                | '*0'

module_list    := module module_list
                | module

module         := simple_declaration
                | vector_declaration
                | string_declaration

simple_declaration := ':' name_id RELOC data_word

vector_declaration := '[' name_id RELOC initial_list '*0'

string_declaration := '"' name_id RELOC length string_id

data_word      := 'g' name_id
                | CONSTANT_VALUE

initial_list   := initializer initial_list
                |

initializer     := data_word
                | '"' string_id
                | '[' initializer initial_list ']'

name_id        := ID

string_id      := ID

length        := CONSTANT_VALUE
```

Function Parsil

```

file                := module_list '*0'
                    | '*0'

module_list         := module module_list
                    | module

module              := external_module
                    | function_module

external_module     := ':' ID

function_module     := header function '*0'

header              := function_name declarations 'H'

function_name       := '(' ID

declarations        := arguments prolog scope_stmts

arguments           := argument arguments
                    |

prolog              := 'P' min_args max_args RELOC

scope_stmts        := scope_statement scope_stmts
                    |

scope_statement     := auto_var_declaration
                    | auto_vector_declaration
                    | auto_string_declaration
                    | extrn_var_declaration

extrn_var_declaration := 'g' REFERENCE name_id

function            := statement_list '*0'

statement_list      := statement statement_list
                    |

statement           := string_definition
                    | twit_string_definition
                    | constant_definition
                    | jump_statement
                    | label_definition

```

```

| extrn_var_declaration
| expression
| return_statement
| word_select
| string_select
| jump_false_statement
| twit_statement
| disable
| enable

string_definition      :=  '"' REFERENCE string_id
twit_string_definition :=  '"' REFERENCE string_id
jump_statement        :=  'J' label
label_definition      :=  'l' label
expression             :=  'E' reference_list '*0'
return_statement      :=  'R' possible_reference_list '*0'
word_select           :=  w_sel_stmt w_case_list end_w_sel
string_select         :=  s_sel_stmt s_case_list end_s_sel
w_sel_stmt            :=  'W' reference_list '*0'
s_sel_stmt            :=  'S' reference_list '*0'
w_case_list           :=  word_case_statement w_case_list
| word_case_statement
str_case_list         :=  str_case_statement str_case_list
| str_case_statement
word_case_statement   :=  word_case statement_list
word_case             :=  simple_word_case
| range_word_case
| default_case
str_case_statement    :=  str_case statement_list
str_case              :=  'b' string_id
| default_case
reference_list         :=  REFERENCE reference_list

```

```

| REFERENCE

possible_reference_list := reference_list
|

jump_false_statement    := 'F' label reference_list '*0'
twit_statement          := 't' reference_list '*0'
argument                := 'a' REFERENCE
end_of_header           := 'H'
auto_var_declaration    := 'A' REFERENCE
auto_vector_declaration := 'V' REFERENCE CONSTANT
auto_string_declaration := 'v' REFERENCE CONSTANT
name_id                 := ID
string_id               := ID
constant_definition     := 'c' REFERENCE CONSTANT
label                   := REFERENCE
end_w_sel               := 'w'
end_s_sel               := 's'
simple_word_case         := 'i' CONSTANT
range_word_case         := 'r' CONSTANT CONSTANT
default_case            := 'D'
disable                 := 'd'
enable                  := 'e'
min_args                := BYTE
max_args                := BYTE

```

C. Eh Grammar

The following is the grammar specification given to the YACC parser generator. From this, YACC produces a parser for Eh. The actions called when productions are recognized have been deleted for brevity.

```
%token  identifier constant string

%token  auto extrn
%token  else if for repeat select while
%token  break goto next return
%token  case default
%token  enable disable twit

%token  '(' '[' '{' ',' ';' ')' ']' '}'

%token  assignop '='
%token  '?' ':'
%token  logor
%token  logand
%token  relationalop
%token  '+' '-'
%token  divop '***'
%token  bitwiseop '&'
%token  preuniop postuniop
%token  colcoln
%token  '#'

%right  assignop '='
%right  '?' ':'

%left  logor
%left  logand
%left  relationalop
%left  '+' '-'
%left  divop '***'
%left  bitwiseop '&'
%left  preuniop postuniop
%%
```

```

IL_program      : list_modules
                 ;
list_modules    : module
                 | list_modules module
                 ;
module          : function
                 | external ';'
                 | error
                 ;
function        : func_name func_header fstatement
                 ;
func_header     : list_of_arguments ')' rbr
                 ;
rbr             :
                 | '#' '(' con_expr ')'
                 | '#' '(' con_expr ',' con_expr ')'
                 ;
func_name       : identifier '('
                 ;
list_of_arguments :
                 | '?'
                 | argument_list
                 | argument_list ';'
                 | ';' argument_list
                 | argument_list ';' argument_list
                 | argument_list ';' '?'
                 ;
argument_list   : identifier
                 | argument_list ',' identifier
                 ;
external        : external_word initial_word
                 | external_ptr_word list_word_data
                 | external_ptr_byte initial_byte
                 ;
external_word    : identifier rbr
                 ;
external_ptr_word : identifier '[' constant... ']' rbr
                 ;
external_ptr_byte : identifier '{' constant... '}' rbr
                 ;
con_expr        : expr
                 ;
constant...     : con_expr
                 |
                 ;
list_word_data  :
                 | ':' list_words

```

```

initial_byte      :
                  | ':' string
                  ;
initial_word      : ':' expr
                  ;
list_words        : list_words ',' word_list_element
                  | word_list_element
                  ;
hook              :
                  ;
word_list_element : expr
                  | '[' hook list_words ']'
                  ;
statement         : label ':' statement
                  | '{' stat_list '}'
                  | compound_statement
                  | simple_statement ';'
                  | ';'
                  | error
                  ;
label             : identifier
                  ;
fstatement        : hend compound_statement
                  | hend simple_statement ';'
                  | hend ';'
                  | '{' fstat_list '}'
                  | error
                  ;
fstat_list        : scope_stmt_list hend stat_list
                  | hend stat_list
                  ;
hend              :
                  ;
scope_stmt_list   : scope_stmt_list scope_stmt ';'
                  | scope_stmt ';'
                  ;
scope_stmt        : auto list_auto_defns
                  | extrn list_extrn
                  ;
list_auto_defns   : auto_defn
                  | list_auto_defns ',' auto_defn
                  ;
auto_defn         : identifier
                  | identifier '[' con_expr ']'
                  | identifier '{' con_expr '}'
                  ;

```

```

list_extrn      : extrn.var
                 | list_extrn ',' extrn.var
                 ;
extrn.var       : identifier
                 ;
stat_list       : statement
                 | stat_list statement
                 ;
simple_statement : transfer_stmt
                 | interrupt_control_stmt
                 | twit_stmt
                 | expr
                 ;
transfer_stmt   : break
                 | next
                 | return '(' expr ')'
                 | return
                 | goto identifier
                 ;
interrupt_control_stmt : enable
                       | disable
                       ;
twit_kwd        : twit
                 ;
twit_stmt       : twit_kwd '(' parm_list ')'
                 ;
compound_statement : then_clause statement
                   | if_then_else statement
                   | while_clause statement
                   | for_clause statement
                   | repeat_clause statement
                   | w_sel_clause '{' w_case.s.list '}'
                   | b_sel_clause '{' b_case.s.list '}'
then_clause     : if '(' expr ')'
                 ;
if_then_else    : then_clause statement else
                 ;
repeat_clause   : repeat
                 ;
while_clause    : while '(' expr ')'
                 ;
for_clause      : for '(' exp. ';' exp. ';' exp. ')'
                 ;
exp.            :
                 | expr
                 ;
w_sel_clause    : select '(' expr ')'
                 ;

```



```

b.sel.clause      : select '{' expr '}'
                  ;
w.case.s.list     : w.case.stat
                  | w.case.s.list w.case.stat
                  ;
b.case.s.list     : b.case.stat
                  | b.case.s.list b.case.stat
                  ;
w.case.stat       : wcase.list statement
                  ;
b.case.stat       : bycase.list statement
                  ;
wcase.list        : case.const.wo ':'
                  | wcase.list case.const.wo ':'
                  ;
bcase.list        : case.const.by ':'
                  | bycase.list case.const.by ':'
                  ;
case.const.wo     : case con_expr
                  | case con_expr colcoln con_expr
                  | case relationalop con_expr
                  | default
                  ;
case.const.by     : case string
                  | default
                  ;
expr              : expr assignop expr
                  | expr '=' expr
                  | expr '?' expr ':' expr
                  | expr logor expr
                  | expr logand expr
                  | expr relationalop expr
                  | expr bitwiseop expr
                  | expr '&' expr
                  | expr '+' expr
                  | expr '-' expr
                  | expr '***' expr
                  | expr divop expr
                  | primary
                  ;
primary           : var
                  | preuniop primary
                  | '&' primary
                  | '***' primary
                  | '-' primary
                  | postuniop primary
                  ;
var               : identifier

```

```

| constant
| string
| '(' expr ')'
| var postuniop
| var '[' expr ']'
| var '{' expr '}'
| var '(' parm_list ')'
;
parm_list
:
| parm_list_nn
;
parm_list_nn
: expr
| parm_list_nn ',' expr
;

```

D. Cleanil Grammar

The grammar of **Cleanil** is similar to the grammar of **Parsil**. There are, however, no **Cleanil** statements for external modules. The important differences between **Parsil** and **Cleanil** are discussed in chapter 5.

```
file                := module_list '*0'
                   | '*0'

module_list         := module module_list
                   | module

module              := function_module

function_module     := header function '*0'

header              := function_name declarations 'H'

function_name       := '(' ID

declarations        := arguments prolog non_arg

arguments           := argument arguments
                   |

non_arg             := scope_stmts non_arg
                   | string_definition non_arg
                   | twit_string_definition non_arg
                   | constant_definition non_arg

prolog              := 'P' min_args max_args RELOC

scope_stmts        := scope_statement scope_stmts
                   |

scope_statement     := auto_var_declaration
                   | auto_vector_declaration
                   | auto_string_declaration
                   | extrn_var_declaration
```

```

extrn_var_declaration := 'g' REFERENCE name_id
function               := statement_list '*0'
statement_list        := statement statement_list
|
statement             := label_definition
| jump_statement
| expression
| return_statement
| word_select
| string_select
| jump_false_statement
| jump_true_statement
| twit_statement
| disable
| enable

string_definition     := '"' REFERENCE string_id
twit_string_definition := ''' REFERENCE string_id
jump_statement        := 'J' label
label_definition      := 'l' label
expression            := 'E' reference_list '*0'
return_statement      := 'R' possible_reference_list '*0'
word_select           := w_sel_stmt w_case_list end_w_sel
string_select         := s_sel_stmt s_case_list end_s_sel
w_sel_stmt            := 'W' reference_list '*0'
s_sel_stmt            := 'S' reference_list '*0'
w_case_list           := word_case_statement w_case_list
| word_case_statement
str_case_list         := str_case_statement str_case_list
| str_case_statement
word_case_statement   := word_case statement_list

```

```

word_case           := simple_word_case
                    | range_word_case
                    | default_case

str_case_statement  := str_case statement_list

str_case            := 'b' string_id
                    | default_case

reference_list       := REFERENCE reference_list
                    | REFERENCE

possible_reference_list := reference_list
                    |

jump_false_statement := 'F' label reference_list '*0'

jump_true_statement  := 'T' label reference_list '*0'

twit_statement       := 't' reference_list '*0'

argument            := 'a' REFERENCE

end_of_header        := 'H'

auto_var_declaration := 'A' REFERENCE

auto_vector_declaration := 'V' REFERENCE CONSTANT

auto_string_declaration := 'v' REFERENCE CONSTANT

name_id             := ID

string_id           := ID

constant_definition := 'c' REFERENCE CONSTANT

label              := REFERENCE

end_w_sel          := 'w'

end_s_sel          := 's'

simple_word_case     := 'i' CONSTANT

range_word_case      := 'r' CONSTANT CONSTANT

default_case        := 'D'

```

```
disable      := 'd'
enable       := 'e'
min_args     := BYTE
max_args     := BYTE
```

E. An Example

To show what **Lex**, **Parse**, and **Clean** do to an Eh program when it is compiled, progressive changes that occur from the source, to the time it is ready for **Data** and **Code** are displayed. In order for the intermediate language representations to be readable, they have been passed through two utility programs which print them in a readable form. These two utility programs, **Plexil** and **Pil**, have been invaluable in debugging the compiler.

E.1 Source Code

The program being compiled is an unspooler for a line printer. It takes files found under a specific node in the file system, and copies them to the line printer.


```

%*/src/fsys/manifest
.Team_root()
{
    extrn Line;
    auto me, it, c;

    .Set_current_node( "%*/lpt" );
    .Select_output( .Open( "$lpt0", "wa" ) );

    repeat
    {
        if( Son_node( Line ) )
        {
            while( !(it=.Open( Line, "rw", 0 )) )
                .Delay( 10 );
            .Select_input( it );
            .Put( '%f' ); .Put( '%n' );
            while( c=.Get() ) .Put( c );
            .Close( it );
            .Remove_node( Line );
            .Flush();
        }
        else .Delay( 30 );
    }
}
Line{33};
.Team_free_list[500]:500;
.Team_root_stack:125;
Son_node( son_path )
{
    auto comp{32}, i;

    .Walk_structure( "@", comp, FD_SON, 0 );
    if( !comp{0} ) return( 0 );

    son_path{0} = '/';
    for( i=0; son_path{i+1}=comp{i}; ++i );

    return( 1 );
}

```

E.2 Output of Lex

This is a readable version of the **Lexil** output generated by **Lex**. The file `*/src/tmp/.srcgjs` is the initial file which is read by **Lex**. The file `*/cmds/eh/ti` contains the environmental manifests. These manifests define such things as bytes-per-word so that program source statements can deal with these concepts in a portable manner. Manifests have already been expanded, but constants are not folded as can be seen in the call to `.Walk_structure`. The source version has one manifest as the third argument to `.Walk_structure`, While the output of Lex has a complicated constant expression. Note that all constants are printed by Plexil in octal.

```
\ Entering file */tmp/.srcgjs on line 0

\ Entering file */cmds/eh/ti on line 0

\ Entering file */tmp/.srcgjs on line 1

\ Entering file */user/./src/unspool on line 0

\ Entering file */src/fsys/manifest on line 0

\ Entering file */user/./src/unspool on line 1

.Team_root()
```

```

{
    extrn Line;
    auto me, it, c;

    .Set_current_node("*/lpt");
    .Select_output(.Open("$lpt", "wa"));

    repeat
    {
        if(Son_node(Line))
        {
            while( !(it=.Open(Line, "rw", 00)).Delay(012);
                .Select_input(it);
                .Put(014);.Put(016);
                while(c=.Get()).Put(c);
                .Close(it);
                .Remove_node(Line);
                .Flush();
            }
            else .Delay(036);
        }
    }
    Line{041};
    .Team_free_list[0764] : 0764;
    .Team_root_stack : 0175;
    Son_node(son_path)
    {
        auto comp{040}, i;

        .Walk_structure("@", comp, ((040/02 + 01) + 02), 00);
        if( !comp{00})return(00);

        son_path{00}=057;
        for(i=00;son_path{i + 01}=comp{i};++i);

        return(01);
    }
}

\ Entering file */tmp/.srcgjs on line 2

```

E.3 Output of Parse

Note that unreachable statements are present and that all functions have to be passed the number of arguments they are called with. Parse assumes that Clean will take care of these problems. Declarations are left where they are found in the source. Expressions are in polish postfix notation. The operator "mark" which is found in most expressions is used to mark the end of a list of arguments to a function.

```

Line{ 33 }  #(15,15) : "";
.Team_free_list[ 500 ]  #(15,15) : 500 ;
.Team_root_stack #(15,15) : 125;
.Team_root( )
    #(15,15)
    extrn Line;
    auto A4;
    auto A6;
    auto A10;
    \ Hend of declarations
    extrn .Set_current_node;
    S14 = "*/lpt";
    mark S14 , .Set_current_node (nargs;
    extrn .Select_output;
    extrn .Open;
    S22 = "$lpt";
    S24 = "wa";
    mark mark S22 , S24 , .Open (nargs , .Select_output
        (nargs;
label26 :
    extrn Son_node;
    if( !( mark Line , Son_node (nargs) ) goto label34;
    S36 = "rw";
    C40 = 0;
label42 :
    if( !( mark Line , S36 , C40 , .Open (nargs A6 = ! ) )
        goto label44;
    extrn .Delay;

```

```

    C50 = 10;
    mark C50 , .Delay (nargs;
    goto label42;
label44 :
    extrn .Select_input;
    mark A6 , .Select_input (nargs;
    extrn .Put;
    C56 = 12;
    mark C56 , .Put (nargs;
    C60 = 14;
    mark C60 , .Put (nargs;
    extrn .Get;
label64 :
    if( !( mark .Get (nargs A10 =) ) goto label66;
    mark A10 , .Put (nargs;
    goto label64;
label66 :
    extrn .Close;
    mark A6 , .Close (nargs;
    extrn .Remove_node;
    mark Line , .Remove_node (nargs;
    extrn .Flush;
    mark .Flush (nargs;
    goto label76;
label34 :
    C100 = 30;
    mark C100 , .Delay (nargs;
label76 :
    goto label26;
label30 :
    return( );
Son_node(arg2 )
    #(15,15)
    auto A_s4{32};
    auto A6;
    \ Hend of declarations
    extrn .Walk_structure;
    S12 = "@";
    C14 = 19;
    C16 = 0;
    mark S12 , A_s4 , C14 , C16 , .Walk_structure (nargs;
    if( !( C16 A_s4 { !} ) goto label20;
    return( C16 );
label20 :
    C22 = 47;
    C22 C16 arg2 { {=;
    C16 A6 =;
    goto label24;

```

```
label26 :  
    A6 ++pre;  
label24 :  
    C32 = 1;  
    if( !( A6 A_s4 { A6 C32 + arg2 { (=) ) goto label30;  
        goto label26;  
label30 :  
    return( C32 );  
    return( );
```

E.4 Output of Clean

Note that unreachable code is removed and that the number of arguments passed to a function is required only for certain functions. All declarations except internal labels have been moved before the end of declarations comment. Word and byte fetch operators have been inserted, and the last if statement in the function `.Son_node` has had the conditional jump reversed.

```
.Team_root( )
    #(15,15)
    extrn Line;
    auto A4;
    auto A6;
    auto A10;
    extrn .Set_current_node;
    S14 = "%/lpt";
    extrn .Select_output;
    extrn .Open;
    S22 = "$lpt";
    S24 = "wa";
    extrn Son_node;
    S36 = "rw";
    C40 = 0;
    extrn .Delay;
    C50 = 10;
    extrn .Select_input;
    extrn .Put;
    C56 = 12;
    C60 = 14;
    extrn .Get;
    extrn .Close;
    extrn .Remove_node;
    extrn .Flush;
    C100 = 30;
    \ end of declarations
    mark S14 , .Set_current_node ( ;
    mark mark S22 , S24 , .Open (nargs , .Select_output
```

```

                                (nargs;
label26 :
    if( !( mark Line , Son_node ( ) ) goto label34;
label42 :
    if( !( mark Line , S36 , C40 , .Open (nargs A6 = ! ) )
        goto label44;
    mark C50 , .Delay (nargs;
    goto label42;
label44 :
    mark A6 , .Select_input (nargs;
    mark C56 , .Put ( ;
    mark C60 , .Put ( ;
label64 :
    if( !( mark .Get ( A10 = ) ) goto label66;
    mark A10 , .Put ( ;
    goto label64;
label66 :
    mark A6 , .Close ( ;
    mark Line , .Remove_node ( ;
    mark .Flush ( ;
    goto label26;
label34 :
    mark C100 , .Delay (nargs;
    goto label26;
Son_node(arg2 )
    #(15,15)
    auto A_s4{32};
    auto A6;
    extrn .Walk_structure;
    S12 = "@";
    C14 = 19;
    C16 = 0;
    C22 = 47;
    C32 = 1;
    \ HEND of declarations
    mark S12 , A_s4 , C14 , C16 , .Walk_structure ( ;
    if( !( A_s4 C16 s_{ By_fet ! } ) goto label20;
    return( C16 );
label20 :
    arg2 C16 s_{ C22 s_{= ;
    A6 C16 s_{= ;
    goto label24;
label26 :
    A6 ++pre;
label24 :
    if( A6 A_s4 { By_fet A6 C32 + arg2 { {= ) goto label26;
    return( C32 );

```